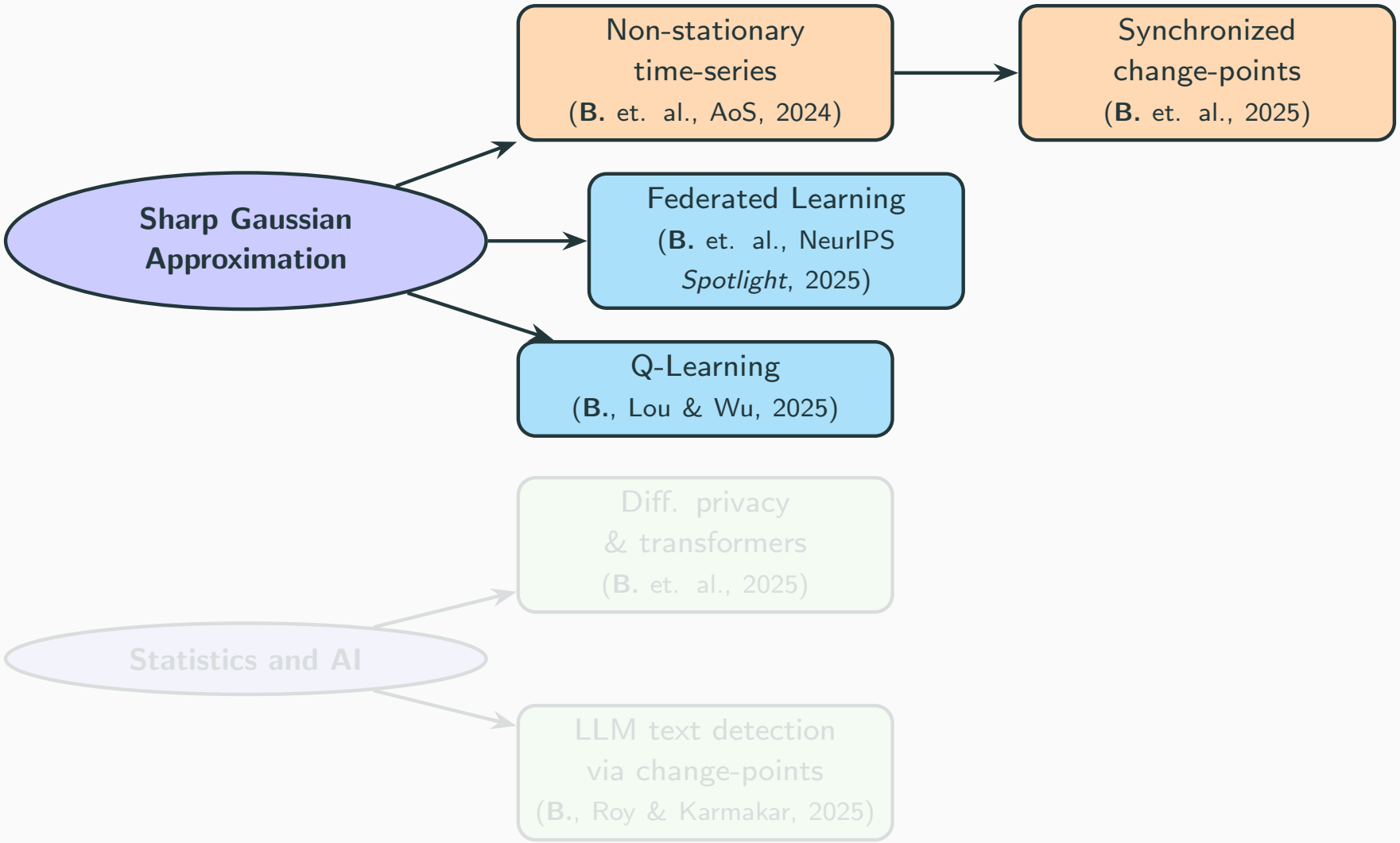
Sharp Gaussian approximations for iterative algorithms

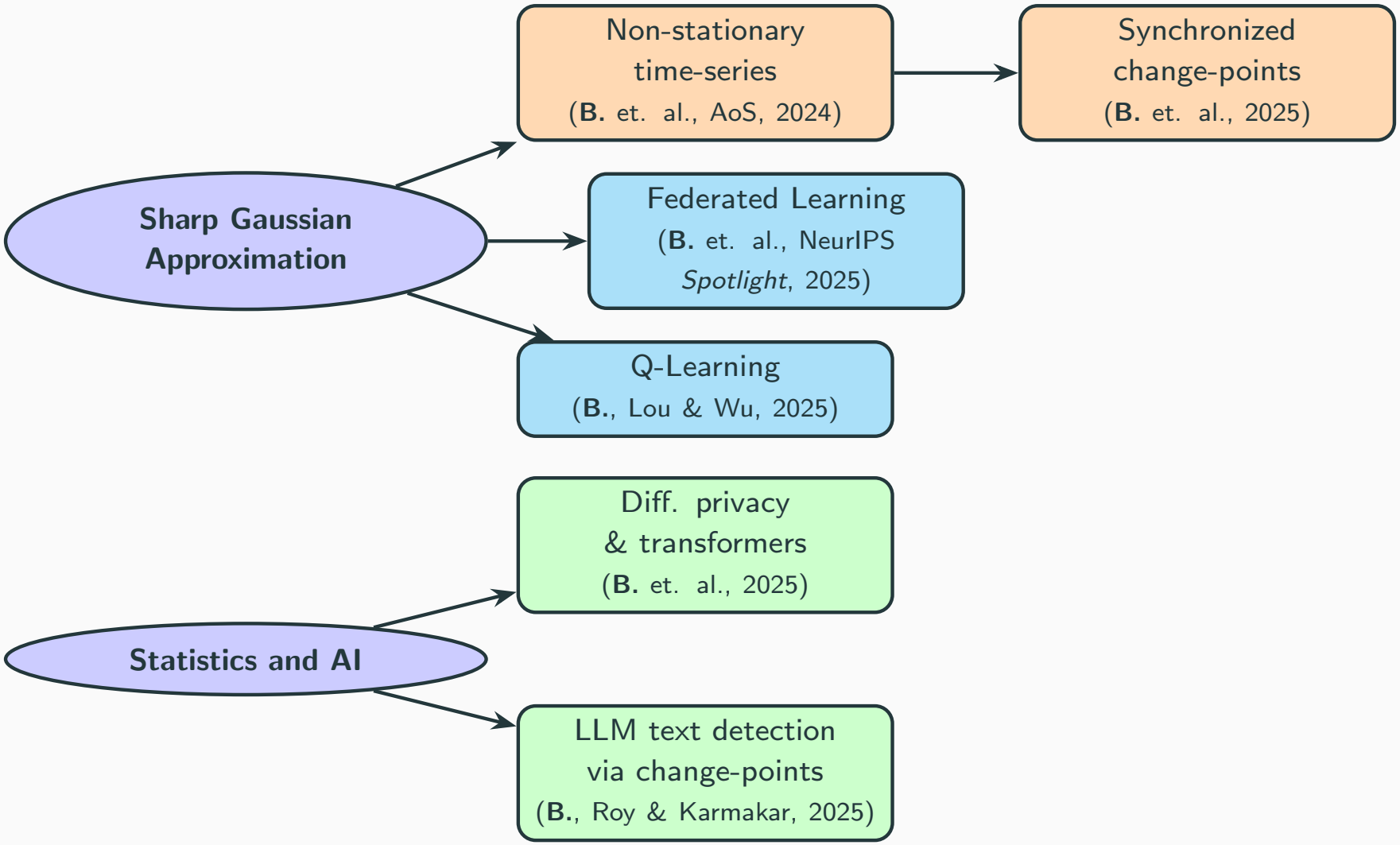
Soham Bonnerjee

Department of Statistics, University of Chicago

Research Overview

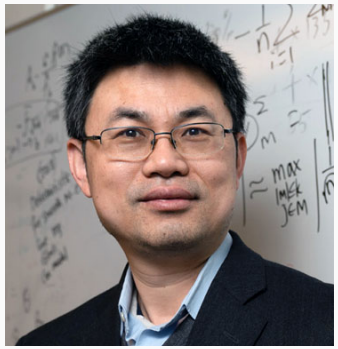


Research Overview



Acknowledgements

Based on joint works with



Wei Biao Wu
University of Chicago



Sayar Karmakar
University of Florida



Zhipeng Lou
UC San Diego



Maggie Cheng
Illinois Institute of Technology

Agenda

- A problem from learning theory: Byzantine attack detection in federated learning. (Blanchard et al., 2017; Yin et al., 2018; Data and Diggavi, 2021; Tu et al., 2021; Qian et al., 2024)
- What iterative algorithm do we use?
- View the iterative algorithms as a time-series.
- Why and how does Gaussian approximation results from time-series literature help us solve the problem?

Agenda

- A problem from learning theory: Byzantine attack detection in federated learning. (Blanchard et al., 2017; Yin et al., 2018; Data and Diggavi, 2021; Tu et al., 2021; Qian et al., 2024)
- What iterative algorithm do we use?
- View the iterative algorithms as a time-series.
- Why and how does Gaussian approximation results from time-series literature help us solve the problem?

Agenda

- A problem from learning theory: Byzantine attack detection in federated learning. (Blanchard et al., 2017; Yin et al., 2018; Data and Diggavi, 2021; Tu et al., 2021; Qian et al., 2024)
- What iterative algorithm do we use?
- View the iterative algorithms as a time-series.
- Why and how does Gaussian approximation results from time-series literature help us solve the problem?

Agenda

- A problem from learning theory: Byzantine attack detection in federated learning. (Blanchard et al., 2017; Yin et al., 2018; Data and Diggavi, 2021; Tu et al., 2021; Qian et al., 2024)
- What iterative algorithm do we use?
- View the iterative algorithms as a time-series.
- Why and how does Gaussian approximation results from time-series literature help us solve the problem?

Preliminaries: Gradient Descent

Goal: $\hat{\theta} = \arg\min_\theta \frac{1}{n} \sum_{i=1}^n f(\theta, \xi_i)$. ξ_i are i.i.d.

Example: for linear regression, $f(\theta, \xi_i) = (y_i - \theta^\top x_i)^2$, $\xi_i = (x_i, y_i)$, $i \in [n]$.

Preliminaries: Gradient Descent

Goal: $\hat{\theta} = \arg\min_\theta \frac{1}{n}\sum_{i=1}^n f(\theta, \xi_i)$. ξ_i are i.i.d.

Example: for linear regression, $f(\theta, \xi_i) = (y_i - \theta^\top x_i)^2$, $\xi_i = (x_i, y_i)$, $i \in [n]$.

Preliminaries: Gradient Descent

Goal: $\hat{\theta} = \arg\min_\theta \frac{1}{n}\sum_{i=1}^n f(\theta, \xi_i)$. ξ_i are i.i.d.

Example: for linear regression, $f(\theta, \xi_i) = (y_i - \theta^\top x_i)^2$, $\xi_i = (x_i, y_i)$, $i \in [n]$.

Gradient Descent:

- $\theta_0 \leftarrow$ initialization; $\{\eta_t\} \leftarrow$ step-size/learning rate.

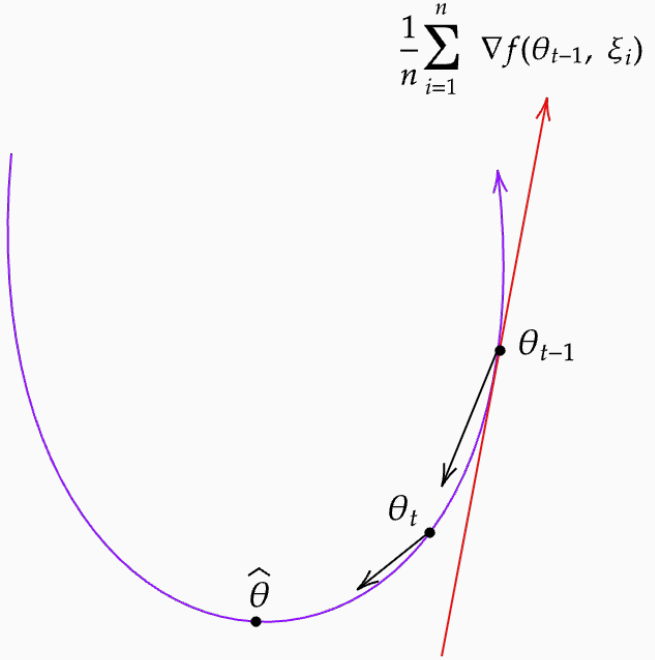
Preliminaries: Gradient Descent

Goal: $\hat{\theta} = \arg\min_\theta \frac{1}{n}\sum_{i=1}^n f(\theta, \xi_i)$, ξ_i i.i.d.

Example: for linear regression, $f(\theta, \xi_i) = (y_i - \theta^\top x_i)^2$, $\xi_i = (x_i, y_i)$, $i \in [n]$.

Gradient Descent:

- $\theta_0 \leftarrow$ initialization; $\{\eta_t\} \leftarrow$ step-size/learning rate.
- Iterate:
  $\theta_t \leftarrow \theta_{t-1} - \eta_t \ \frac{1}{n}\sum_{i=1}^n \nabla_\theta f(\theta_{t-1}, \xi_i)$,
  $t = 1, 2, \ldots$.
- Drawback: Offline; n gradient calculations at every step.



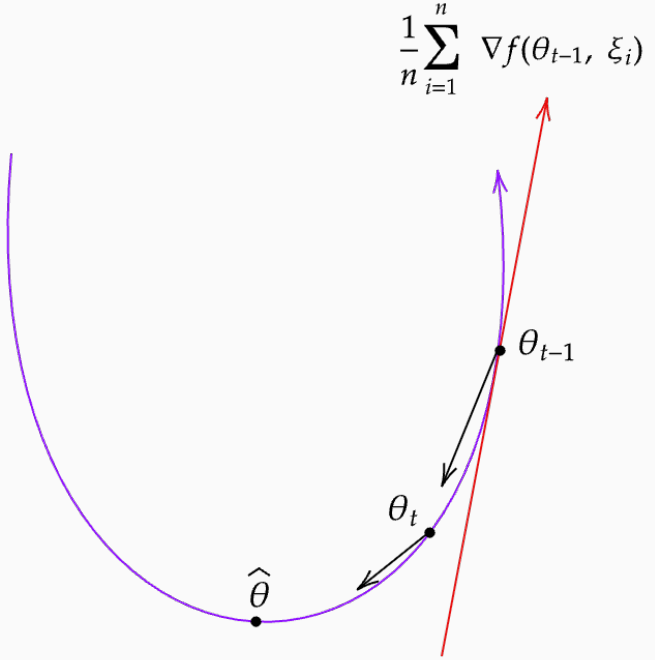
Preliminaries: Gradient Descent

Goal: $\hat{\theta} = \arg\min_\theta \frac{1}{n}\sum_{i=1}^n f(\theta, \xi_i)$, ξ_i i.i.d.

Example: for linear regression, $f(\theta, \xi_i) = (y_i - \theta^\top x_i)^2$, $\xi_i = (x_i, y_i)$, $i \in [n]$.

Gradient Descent:

- $\theta_0 \leftarrow$ initialization; $\{\eta_t\} \leftarrow$ step-size/learning rate.
- Iterate:
  $\theta_t \leftarrow \theta_{t-1} - \eta_t \ \frac{1}{n}\sum_{i=1}^n \nabla_\theta f(\theta_{t-1}, \xi_i)$,
  $t = 1, 2, \ldots$.
- Drawback: Offline; n gradient calculations at every step.



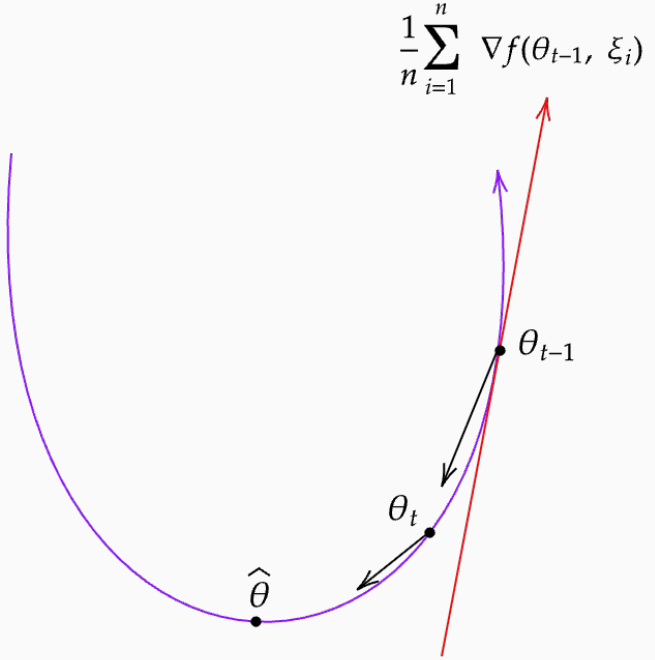
Preliminaries: Gradient Descent

Goal: $\hat{\theta} = \arg\min_\theta \frac{1}{n}\sum_{i=1}^n f(\theta, \xi_i)$, ξ_i i.i.d.

Example: for linear regression, $f(\theta, \xi_i) = (y_i - \theta^\top x_i)^2$, $\xi_i = (x_i, y_i)$, $i \in [n]$.

Gradient Descent:

- $\theta_0 \leftarrow$ initialization; $\{\eta_t\} \leftarrow$ step-size/learning rate.
- Iterate:
  $\theta_t \leftarrow \theta_{t-1} - \eta_t \ \frac{1}{n}\sum_{i=1}^n \nabla_\theta f(\theta_{t-1}, \xi_i)$,
  $t = 1, 2, \ldots$.
- Drawback: Offline; n gradient calculations at every step.



Preliminaries: Stochastic Gradient Descent

Goal: $\hat{\theta} = \arg\min_\theta \frac{1}{n}\sum_{i=1}^n f(\theta, \xi_i)$.

Stochastic Gradient Descent(SGD):

- Iterate: $\theta_t \leftarrow \theta_{t-1} - \eta_t \nabla_\theta f(\theta_{t-1}, \xi_t)$, $t = 1, 2, \ldots, n$.
- Online; one gradient calculation per iterate.
- Workhorse for modern optimization, and has many variants!! (Poljak, 1964; Nesterov, 1983; Duchi, Hazan, and Singer, 2011; Kingma, 2014; LeCun et al., 2002; Bottou, 2010).

Focus of this talk: **Local SGD** (Stich, 2019; Li et al., 2020; Yang et al., 2019; He, Jin, and J. Chen, 2021 etc); used in **Federated Learning**.

Preliminaries: Stochastic Gradient Descent

Goal: $\hat{\theta} = \arg\min_\theta \frac{1}{n}\sum_{i=1}^n f(\theta, \xi_i)$.

Stochastic Gradient Descent(SGD):

- Iterate: $\theta_t \leftarrow \theta_{t-1} - \eta_t \nabla_\theta f(\theta_{t-1}, \xi_t)$, $t = 1, 2, \ldots, n$.
- Online; one gradient calculation per iterate.
- Workhorse for modern optimization, and has many variants!! (Poljak, 1964; Nesterov, 1983; Duchi, Hazan, and Singer, 2011; Kingma, 2014; LeCun et al., 2002; Bottou, 2010).

Focus of this talk: **Local SGD** (Stich, 2019; Li et al., 2020; Yang et al., 2019; He, Jin, and J. Chen, 2021 etc); used in **Federated Learning**.

Preliminaries: Stochastic Gradient Descent

Goal: $\hat{\theta} = \arg\min_\theta \frac{1}{n}\sum_{i=1}^n f(\theta, \xi_i)$.

Stochastic Gradient Descent(SGD):

- Iterate: $\theta_t \leftarrow \theta_{t-1} - \eta_t \nabla_\theta f(\theta_{t-1}, \xi_t),\ t = 1, 2, \ldots, n.$
- Online; one gradient calculation per iterate.
- Workhorse for modern optimization, and has many variants!! (Poljak, 1964; Nesterov, 1983; Duchi, Hazan, and Singer, 2011; Kingma, 2014; LeCun et al., 2002; Bottou, 2010).

Focus of this talk: **Local SGD** (Stich, 2019; Li et al., 2020; Yang et al., 2019; He, Jin, and J. Chen, 2021 etc); used in **Federated Learning**.

Preliminaries: Stochastic Gradient Descent

Goal: $\hat{\theta} = \arg\min_\theta \frac{1}{n}\sum_{i=1}^n f(\theta, \xi_i)$.

Stochastic Gradient Descent(SGD):

- Iterate: $\theta_t \leftarrow \theta_{t-1} - \eta_t \nabla_\theta f(\theta_{t-1}, \xi_t),\ t = 1, 2, \ldots, n.$
- Online; one gradient calculation per iterate.
- Workhorse for modern optimization, and has many variants!! (Poljak, 1964; Nesterov, 1983; Duchi, Hazan, and Singer, 2011; Kingma, 2014; LeCun et al., 2002; Bottou, 2010).

Focus of this talk: **Local SGD** (Stich, 2019; Li et al., 2020; Yang et al., 2019; He, Jin, and J. Chen, 2021 etc); used in **Federated Learning**.

Preliminaries: Stochastic Gradient Descent

Goal: $\hat{\theta} = \arg\min_\theta \frac{1}{n}\sum_{i=1}^n f(\theta, \xi_i)$.

Stochastic Gradient Descent(SGD):

- Iterate: $\theta_t \leftarrow \theta_{t-1} - \eta_t \nabla_\theta f(\theta_{t-1}, \xi_t),\ t = 1, 2, \ldots, n.$
- Online; one gradient calculation per iterate.
- Workhorse for modern optimization, and has many variants!! (Poljak, 1964; Nesterov, 1983; Duchi, Hazan, and Singer, 2011; Kingma, 2014; LeCun et al., 2002; Bottou, 2010).

Focus of this talk: **Local SGD** (Stich, 2019; Li et al., 2020; Yang et al., 2019; He, Jin, and J. Chen, 2021 etc); used in **Federated Learning**.

Preliminaries: Stochastic Gradient Descent

Goal: $\hat{\theta} = \arg\min_\theta \frac{1}{n}\sum_{i=1}^n f(\theta, \xi_i)$.

Stochastic Gradient Descent(SGD):

- Iterate: $\theta_t \leftarrow \theta_{t-1} - \eta_t \nabla_\theta f(\theta_{t-1}, \xi_t),\ t = 1, 2, \ldots, n.$
- Online; one gradient calculation per iterate.
- Workhorse for modern optimization, and has many variants!! (Poljak, 1964; Nesterov, 1983; Duchi, Hazan, and Singer, 2011; Kingma, 2014; LeCun et al., 2002; Bottou, 2010).

Focus of this talk: **Local SGD** (Stich, 2019; Li et al., 2020; Yang et al., 2019; He, Jin, and J. Chen, 2021 etc); used in **Federated Learning**.

Preliminaries: Stochastic Gradient Descent

Goal: $\hat{\theta} = \arg\min_\theta \frac{1}{n}\sum_{i=1}^n f(\theta, \xi_i)$.

Stochastic Gradient Descent(SGD):

- Iterate: $\theta_t \leftarrow \theta_{t-1} - \eta_t \nabla_\theta f(\theta_{t-1}, \xi_t),\ t = 1, 2, \ldots, n.$
- Online; one gradient calculation per iterate.
- Workhorse for modern optimization, and has many variants!! (Poljak, 1964; Nesterov, 1983; Duchi, Hazan, and Singer, 2011; Kingma, 2014; LeCun et al., 2002; Bottou, 2010).

Focus of this talk: **Local SGD** (Stich, 2019; Li et al., 2020; Yang et al., 2019; He, Jin, and J. Chen, 2021 etc); used in **Federated Learning**.

Federated Learning

- Popularized by McMahan et al., 2017; Karimireddy et al., 2020.
- Other (extensive) uses: Autonomous vehicle control/ Internet-of-Vehicles (M. Chen and Cui, 2024), health event prediction (Y. Chen et al., 2020), Internet-of-Things: IoT (Du et al., 2022; Fan, Huang, and Liu, 2022) etc.

Federated Learning

An IoT example simplified from Apple, Ji et al., 2025:



Client 1

$\xi_1^1, \xi_2^1, \ldots, \xi_n^1 \sim P_1$

Local Objective: $f_1(\theta, \xi^1)$



Client 2

$\xi_1^2, \xi_2^2, \ldots, \xi_n^2 \sim P_2$

Local Objective: $f_2(\theta, \xi^2)$



Client 4

$\xi_1^4, \xi_2^4, \ldots, \xi_n^4 \sim P_4$

Local Objective: $f_4(\theta, \xi^4)$



Client 3

$\xi_1^3, \xi_2^3, \ldots, \xi_n^3 \sim P_3$

Local Objective: $f_3(\theta, \xi^3)$

- The objectives f_k's can all be same, eg. keyword prediction.
- The servers/clients collaborate to fit a consensus objective function:

$$\hat{\theta} = \arg\min_{\theta} \sum_{k=1}^{K} \frac{w_k}{n} \sum_{i=1}^{n} f_k(\theta, \xi_i^k), \quad \sum_{k=1}^{K} w_k = 1.$$

Federated Learning

An IoT example simplified from Apple, Ji et al., 2025:



Client 1

$\xi_1^1, \xi_2^1, \ldots, \xi_n^1 \sim P_1$

Local Objective: $f_1(\theta, \xi^1)$



Client 2

$\xi_1^2, \xi_2^2, \ldots, \xi_n^2 \sim P_2$

Local Objective: $f_2(\theta, \xi^2)$



Client 4

$\xi_1^4, \xi_2^4, \ldots, \xi_n^4 \sim P_4$

Local Objective: $f_4(\theta, \xi^4)$



Client 3

$\xi_1^3, \xi_2^3, \ldots, \xi_n^3 \sim P_3$

Local Objective: $f_3(\theta, \xi^3)$

- The objectives f_k's can all be same, eg. keyword prediction.
- The servers/clients collaborate to fit a consensus objective function:

$$\hat{\theta} = \arg\min_{\theta} \sum_{k=1}^{K} \frac{w_k}{n} \sum_{i=1}^{n} f_k(\theta, \xi_i^k), \ \sum_{k=1}^{K} w_k = 1.$$

Federated Learning

An IoT example simplified from Apple, Ji et al., 2025:



Client 1

$\xi_1^1, \xi_2^1, \ldots, \xi_n^1 \sim P_1$

Local Objective: $f_1(\theta, \xi^1)$



Client 2

$\xi_1^2, \xi_2^2, \ldots, \xi_n^2 \sim P_2$

Local Objective: $f_2(\theta, \xi^2)$



Client 4

$\xi_1^4, \xi_2^4, \ldots, \xi_n^4 \sim P_4$

Local Objective: $f_4(\theta, \xi^4)$



Client 3

$\xi_1^3, \xi_2^3, \ldots, \xi_n^3 \sim P_3$

Local Objective: $f_3(\theta, \xi^3)$

- The objectives f_k's can all be same, eg. keyword prediction.
- The servers/clients collaborate to fit a consensus objective function:

$$\hat{\theta} = \arg\min_{\theta} \sum_{k=1}^{K} \frac{w_k}{n} \sum_{i=1}^{n} f_k(\theta, \xi_i^k), \ \sum_{k=1}^{K} w_k = 1.$$

SGD and Federated Learning

Goal: $\hat{\theta} = \arg\min_\theta \sum_{k=1}^K \frac{w_k}{n} \sum_{i=1}^n f_k(\theta_{t-1}, \xi_i^k)$.

Can we run an SGD?

- Run an SGD:

$$\theta_t = \theta_{t-1} - \eta_t \sum_{k=1}^K w_k \nabla_\theta f_k(\theta, \xi_i^k).$$

Drawbacks:

- Typically $K \approx n^c$ for some $c > 0$, so computationally expensive.
- Combining gradients of two distant networks is costly and not too helpful.

What is the remedy?: parallellization!

SGD and Federated Learning

Goal: $\hat{\theta} = \arg\min_\theta \sum_{k=1}^K \frac{w_k}{n} \sum_{i=1}^n f_k(\theta_{t-1}, \xi_i^k)$.

Can we run an SGD?

- Run an SGD:

$$\theta_t = \theta_{t-1} - \eta_t \sum_{k=1}^K w_k \nabla_\theta f_k(\theta, \xi_i^k).$$

Drawbacks:

- Typically $K \approx n^c$ for some $c > 0$, so computationally expensive.
- Combining gradients of two distant networks is costly and not too helpful.

What is the remedy?: parallellization!

SGD and Federated Learning

Goal: $\hat{\theta} = \arg\min_\theta \sum_{k=1}^K \frac{w_k}{n} \sum_{i=1}^n f_k(\theta_{t-1}, \xi_i^k)$.

Can we run an SGD?

- Run an SGD:

$$\theta_t = \theta_{t-1} - \eta_t \sum_{k=1}^K w_k \nabla_\theta f_k(\theta, \xi_i^k).$$

Drawbacks:

- Typically $K \approx n^c$ for some $c > 0$, so computationally expensive.
- Combining gradients of two distant networks is costly and not too helpful.

What is the remedy?: parallellization!

SGD and Federated Learning

Goal: $\hat{\theta} = \arg\min_\theta \sum_{k=1}^K \frac{w_k}{n} \sum_{i=1}^n f_k(\theta_{t-1}, \xi_i^k)$.

Can we run an SGD?

- Run an SGD:

$$\theta_t = \theta_{t-1} - \eta_t \sum_{k=1}^K w_k \nabla_\theta f_k(\theta, \xi_i^k).$$

Drawbacks:

- Typically $K \approx n^c$ for some $c > 0$, so computationally expensive.
- Combining gradients of two distant networks is costly and not too helpful.

What is the remedy?: parallellization!

SGD and Federated Learning

Goal: $\hat{\theta} = \arg\min_\theta \sum_{k=1}^K \frac{w_k}{n} \sum_{i=1}^n f_k(\theta_{t-1}, \xi_i^k)$.

Can we run an SGD?

- Run an SGD:

$$\theta_t = \theta_{t-1} - \eta_t \sum_{k=1}^K w_k \nabla_\theta f_k(\theta, \xi_i^k).$$

Drawbacks:

- Typically $K \approx n^c$ for some $c > 0$, so computationally expensive.
- Combining gradients of two distant networks is costly and not too helpful.

What is the remedy?: parallellization!

SGD and Federated Learning

Goal: $\hat{\theta} = \arg\min_\theta \sum_{k=1}^K \frac{w_k}{n} \sum_{i=1}^n f_k(\theta_{t-1}, \xi_i^k)$.

Can we run an SGD?

- Run an SGD:

$$\theta_t = \theta_{t-1} - \eta_t \sum_{k=1}^K w_k \nabla_\theta f_k(\theta, \xi_i^k).$$

Drawbacks:

- Typically $K \approx n^c$ for some $c > 0$, so computationally expensive.
- Combining gradients of two distant networks is costly and not too helpful.

What is the remedy?: parallellization!

Local SGD

- Each client k can run their own *parallel*, SGD:

$$\theta_t^k = \theta_{t-1}^k - \eta_t \nabla f_k(\theta_{t-1}^k, \xi_t^k).$$

- They also must collaborate $\Rightarrow$ share gradient with *neighbors*, *periodically*.
- Known as Local SGD: Stich, 2019; Koloskova et al., 2020 etc.
- **However, this has a potential to be hacked!**

Local SGD

- Each client k can run their own *parallel*, SGD:

$$\theta_t^k = \theta_{t-1}^k - \eta_t \nabla f_k(\theta_{t-1}^k, \xi_t^k).$$

- They also must collaborate $\Rightarrow$ share gradient with *neighbors*, *periodically*.
- Known as Local SGD: Stich, 2019; Koloskova et al., 2020 etc.
- **However, this has a potential to be hacked!**

Local SGD

- Each client k can run their own *parallel*, SGD:

$$\theta_t^k = \theta_{t-1}^k - \eta_t \nabla f_k(\theta_{t-1}^k, \xi_t^k).$$

- They also must collaborate $\Rightarrow$ share gradient with *neighbors*, *periodically*.
- Known as Local SGD: Stich, 2019; Koloskova et al., 2020 etc.
- **However, this has a potential to be hacked!**

Local SGD

- Each client k can run their own *parallel*, SGD:

$$\theta_t^k = \theta_{t-1}^k - \eta_t \nabla f_k(\theta_{t-1}^k, \xi_t^k).$$

- They also must collaborate $\Rightarrow$ share gradient with *neighbors*, *periodically*.
- Known as Local SGD: Stich, 2019; Koloskova et al., 2020 etc.
- **However, this has a potential to be hacked!**

Local SGD

- Each client k can run their own *parallel*, SGD:

$$\theta_t^k = \theta_{t-1}^k - \eta_t \nabla f_k(\theta_{t-1}^k, \xi_t^k).$$

- They also must collaborate $\Rightarrow$ share gradient with *neighbors*, *periodically*.
- Known as Local SGD: Stich, 2019; Koloskova et al., 2020 etc.
- **However, this has a potential to be hacked!**

Byzantine Attack on Local SGD

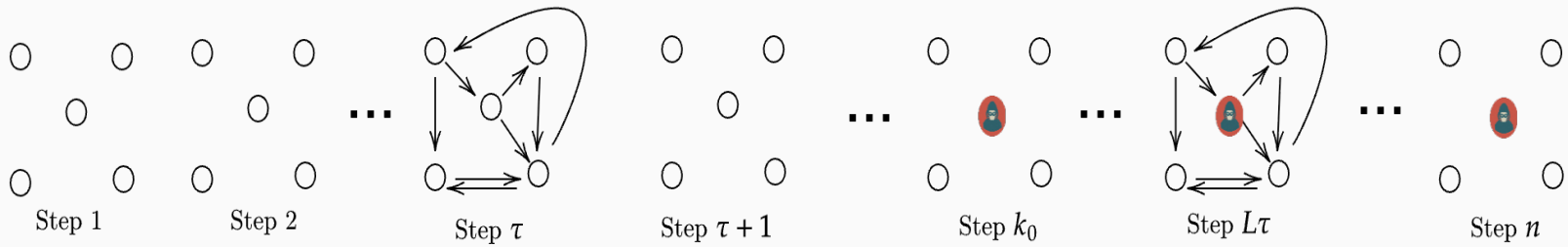


Figure 1: Byzantine Attack : Yin et al., 2018; Data and Diggavi, 2021, etc

- **Goal 1**: stop the collaborative learning system ASAP. (*Early stopping*).
  Need: A sequential testing procedure for existence of change-point.
- **Goal 2**: identify when the attack took place. (*Attack instance detection*).
  Need: Estimate/localize change-point once you stop.

Byzantine Attack on Local SGD

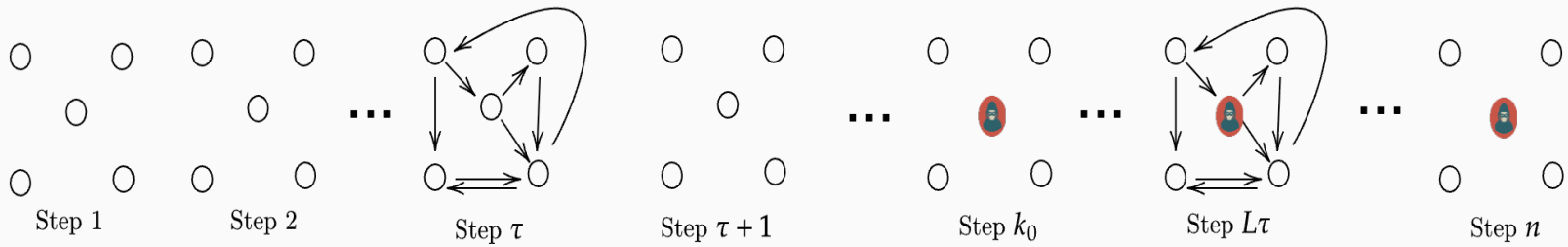


Figure 1: Byzantine Attack : Yin et al., 2018; Data and Diggavi, 2021, etc

- **Goal 1**: stop the collaborative learning system ASAP. (*Early stopping*).
  Need: A sequential testing procedure for existence of change-point.
- **Goal 2**: identify when the attack took place. (*Attack instance detection*).
  Need: Estimate/localize change-point once you stop.

Byzantine Attack on Local SGD

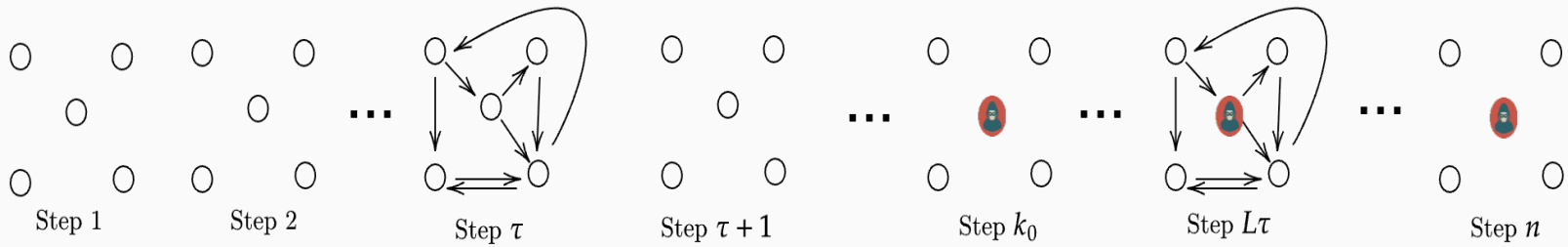


Figure 1: Byzantine Attack : Yin et al., 2018; Data and Diggavi, 2021, etc

- **Goal 1**: stop the collaborative learning system ASAP. (*Early stopping*).
  Need: A sequential testing procedure for existence of change-point.
- **Goal 2**: identify when the attack took place. (*Attack instance detection*).
  Need: Estimate/localize change-point once you stop.

Byzantine Attack on Local SGD

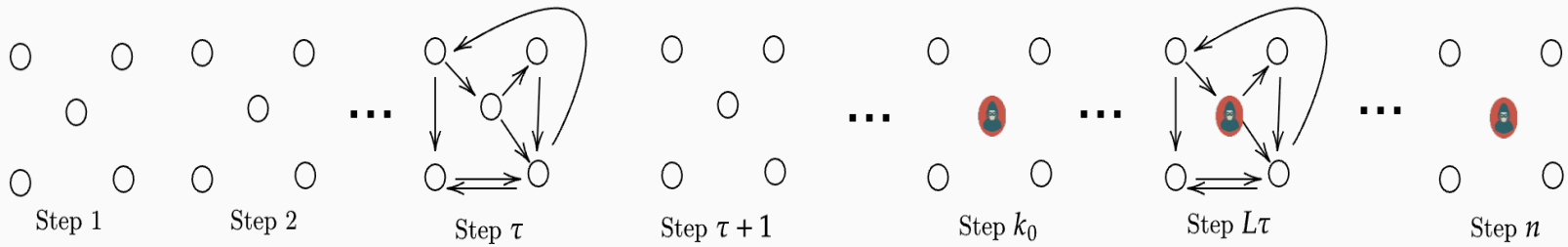


Figure 1: Byzantine Attack : Yin et al., 2018; Data and Diggavi, 2021, etc

- **Goal 1**: stop the collaborative learning system ASAP. (*Early stopping*).
  Need: A sequential testing procedure for existence of change-point.
- **Goal 2**: identify when the attack took place. (*Attack instance detection*).
  Need: Estimate/localize change-point once you stop.

Recap and Way Forward

- **SGD**: an online learning algorithm to optimize differentiable objective functions.
- **Local SGD**: clients parallelly run SGD while periodically sharing data to neighbors.
- **Question**: What if a Byzantine attack changes the objective function of some clients?
- Answer: a sequential analysis framework for testing and estimation.
- *Probabilistic requirement*: Establish a control over the entire local SGD trajectory across clients.

Recap and Way Forward

- **SGD**: an online learning algorithm to optimize differentiable objective functions.
- **Local SGD**: clients parallelly run SGD while periodically sharing data to neighbors.
- **Question**: What if a Byzantine attack changes the objective function of some clients?
- Answer: a sequential analysis framework for testing and estimation.
- *Probabilistic requirement*: Establish a control over the entire local SGD trajectory across clients.

Recap and Way Forward

- **SGD**: an online learning algorithm to optimize differentiable objective functions.
- **Local SGD**: clients parallelly run SGD while periodically sharing data to neighbors.
- **Question**: What if a Byzantine attack changes the objective function of some clients?
- Answer: a sequential analysis framework for testing and estimation.
- *Probabilistic requirement*: Establish a control over the entire local SGD trajectory across clients.

Recap and Way Forward

- **SGD**: an online learning algorithm to optimize differentiable objective functions.
- **Local SGD**: clients parallelly run SGD while periodically sharing data to neighbors.
- **Question**: What if a Byzantine attack changes the objective function of some clients?
- Answer: a sequential analysis framework for testing and estimation.
- *Probabilistic requirement*: Establish a control over the entire local SGD trajectory across clients.

Recap and Way Forward

- **SGD**: an online learning algorithm to optimize differentiable objective functions.
- **Local SGD**: clients parallelly run SGD while periodically sharing data to neighbors.
- **Question**: What if a Byzantine attack changes the objective function of some clients?
- Answer: a sequential analysis framework for testing and estimation.
- *Probabilistic requirement*: Establish a control over the entire local SGD trajectory across clients.

Characterizing uniform control

Take a step-back from local SGD to usual SGD.

- SGD iterates: $\theta_t \leftarrow \theta_{t-1} - \eta_t \nabla_\theta f(\theta_{t-1}, \xi_t)$, $t = 1, 2, \ldots, n$.
- **Uniform control over SGD**: Can we find Gaussian $Z_1, Z_2, \ldots, Z_n$, with *known/estimable* distribution structure, such that

$$\sum_{t=1}^{i} \theta_t \approx \sum_{t=1}^{i} Z_t \text{ for all } i \in [n].$$

- For example: functional CLT for θ_t's (*if it exists*): $\{Z_1, \ldots, Z_n\}$ are jointly Gaussian.
- If yes, we can asymptotically approximate the cut-offs and CIs for sequential inference.

Characterizing uniform control

Take a step-back from local SGD to usual SGD.

- SGD iterates: $\theta_t \leftarrow \theta_{t-1} - \eta_t \nabla_\theta f(\theta_{t-1}, \xi_t)$, $t = 1, 2, \ldots, n$.
- **Uniform control over SGD**: Can we find Gaussian $Z_1, Z_2, \ldots, Z_n$, with *known/estimable* distribution structure, such that

$$\sum_{t=1}^{i} \theta_t \approx \sum_{t=1}^{i} Z_t \text{ for all } i \in [n].$$

- For example: functional CLT for θ_t's (*if it exists*): $\{Z_1, \ldots, Z_n\}$ are jointly Gaussian.
- If yes, we can asymptotically approximate the cut-offs and CIs for sequential inference.

Characterizing uniform control

Take a step-back from local SGD to usual SGD.

- SGD iterates: $\theta_t \leftarrow \theta_{t-1} - \eta_t \nabla_\theta f(\theta_{t-1}, \xi_t)$, $t = 1, 2, \ldots, n$.
- **Uniform control over SGD**: Can we find Gaussian $Z_1, Z_2, \ldots, Z_n$, with *known/estimable* distribution structure, such that

$$\sum_{t=1}^{i} \theta_t \approx \sum_{t=1}^{i} Z_t \text{ for all } i \in [n].$$

- For example: functional CLT for θ_t's (*if it exists*): $\{Z_1, \ldots, Z_n\}$ are jointly Gaussian.
- If yes, we can asymptotically approximate the cut-offs and CIs for sequential inference.

Characterizing uniform control

Take a step-back from local SGD to usual SGD.

- SGD iterates: $\theta_t \leftarrow \theta_{t-1} - \eta_t \nabla_\theta f(\theta_{t-1}, \xi_t)$, $t = 1, 2, \ldots, n$.
- **Uniform control over SGD**: Can we find Gaussian $Z_1, Z_2, \ldots, Z_n$, with *known/estimable* distribution structure, such that

$$\sum_{t=1}^{i} \theta_t \approx \sum_{t=1}^{i} Z_t \text{ for all } i \in [n].$$

- For example: functional CLT for θ_t's (*if it exists*): $\{Z_1, \ldots, Z_n\}$ are jointly Gaussian.
- If yes, we can asymptotically approximate the cut-offs and CIs for sequential inference.

Characterizing uniform control

Take a step-back from local SGD to usual SGD.

- SGD iterates: $\theta_t \leftarrow \theta_{t-1} - \eta_t \nabla_\theta f(\theta_{t-1}, \xi_t)$, $t = 1, 2, \ldots, n$.
- **Uniform control over SGD**: Can we find Gaussian $Z_1, Z_2, \ldots, Z_n$, with *known/estimable* distribution structure, such that

$$\sum_{t=1}^{i} \theta_t \approx \sum_{t=1}^{i} Z_t \text{ for all } i \in [n].$$

- For example: functional CLT for θ_t's (*if it exists*): $\{Z_1, \ldots, Z_n\}$ are jointly Gaussian.
- If yes, we can asymptotically approximate the cut-offs and CIs for sequential inference.

Characterizing uniform control for SGD

Exactly what properties of SGD iterates can we exploit to get such results?

SGD to non-stationarity

SGD iterates: $\theta_t \leftarrow \theta_{t-1} - \eta_t \nabla_\theta f(\theta_{t-1}, \xi_t),\ t = 1, 2, \ldots, n.$

- θ_t is autoregressive in structure; but depends on step-size $\{\eta_t\}$, which may change with t.
- Therefore: $\theta_t = H_t(\xi_t, \xi_{t-1}, \ldots, \xi_1)$ $\leftarrow$ non-stationary time-series.
- In general, model non-stationary processes $\{X_i\}_{i \in \mathbb{Z}}$ as :

$$X_i = H_i(e_i, e_{i-1}, \ldots),\ \{e_i\}_{i \in \mathbb{Z}} \text{ i.i.d. "innovations" or "generators".}$$

Here, $H_i : \mathbb{R}^\infty \to \mathbb{R}$ mbl.

Examples beyond SGD: ARIMA, GARCH, Volterra, Cyclostationary processes etc.

SGD to non-stationarity

SGD iterates: $\theta_t \leftarrow \theta_{t-1} - \eta_t \nabla_\theta f(\theta_{t-1}, \xi_t)$, $t = 1, 2, \ldots, n$.

- θ_t is autoregressive in structure; but depends on step-size $\{\eta_t\}$, which may change with t.
- Therefore: $\theta_t = H_t(\xi_t, \xi_{t-1}, \ldots, \xi_1)$ $\leftarrow$ non-stationary time-series.
- In general, model non-stationary processes $\{X_i\}_{i \in \mathbb{Z}}$ as :

$$X_i = H_i(e_i, e_{i-1}, \ldots), \ \{e_i\}_{i \in \mathbb{Z}} \text{ i.i.d. "innovations" or "generators".}$$

Here, $H_i : \mathbb{R}^\infty \to \mathbb{R}$ mbl.

Examples beyond SGD: ARIMA, GARCH, Volterra, Cyclostationary processes etc.

SGD to non-stationarity

SGD iterates: $\theta_t \leftarrow \theta_{t-1} - \eta_t \nabla_\theta f(\theta_{t-1}, \xi_t),\ t = 1, 2, \ldots, n.$

- θ_t is autoregressive in structure; but depends on step-size $\{\eta_t\}$, which may change with t.
- Therefore: $\theta_t = H_t(\xi_t, \xi_{t-1}, \ldots, \xi_1)$ $\leftarrow$ non-stationary time-series.
- In general, model non-stationary processes $\{X_i\}_{i \in \mathbb{Z}}$ as :

$$X_i = H_i(e_i, e_{i-1}, \ldots),\ \{e_i\}_{i \in \mathbb{Z}} \text{ i.i.d. "innovations" or "generators".}$$

Here, $H_i : \mathbb{R}^\infty \to \mathbb{R}$ mbl.

Examples beyond SGD: ARIMA, GARCH, Volterra, Cyclostationary processes etc.

SGD to non-stationarity

SGD iterates: $\theta_t \leftarrow \theta_{t-1} - \eta_t \nabla_\theta f(\theta_{t-1}, \xi_t)$, $t = 1, 2, \ldots, n$.

- θ_t is autoregressive in structure; but depends on step-size $\{\eta_t\}$, which may change with t.
- Therefore: $\theta_t = H_t(\xi_t, \xi_{t-1}, \ldots, \xi_1)$ $\leftarrow$ non-stationary time-series.
- In general, model non-stationary processes $\{X_i\}_{i \in \mathbb{Z}}$ as :

$$X_i = H_i(e_i, e_{i-1}, \ldots), \ \{e_i\}_{i \in \mathbb{Z}} \text{ i.i.d. "innovations" or "generators".}$$

Here, $H_i : \mathbb{R}^\infty \to \mathbb{R}$ mbl.

Examples beyond SGD: ARIMA, GARCH, Volterra, Cyclostationary processes etc.

SGD to non-stationarity

SGD iterates: $\theta_t \leftarrow \theta_{t-1} - \eta_t \nabla_\theta f(\theta_{t-1}, \xi_t),\ t = 1, 2, \ldots, n.$

- θ_t is autoregressive in structure; but depends on step-size $\{\eta_t\}$, which may change with t.
- Therefore: $\theta_t = H_t(\xi_t, \xi_{t-1}, \ldots, \xi_1)$ $\leftarrow$ non-stationary time-series.
- In general, model non-stationary processes $\{X_i\}_{i \in \mathbb{Z}}$ as :

$$X_i = H_i(e_i, e_{i-1}, \ldots),\ \{e_i\}_{i \in \mathbb{Z}} \text{ i.i.d. "innovations" or "generators".}$$

Here, $H_i : \mathbb{R}^\infty \to \mathbb{R}$ mbl.

Examples beyond SGD: ARIMA, GARCH, Volterra, Cyclostationary processes etc.

Zooming out: the key general question

To start with, focus on general non-stationary process.

Questions:

- Find Gaussian $Z_1, \ldots, Z_n$ such that $\max_{1 \le i \le n} |\sum_{j=1}^{i} (X_i - Z_i)| \approx$ small with optimal rate.
- Understand the exact covariance structure of $Z_1, \ldots, Z_n$.

Non-stationary structure$\approx$ arbitrary dependence! **For motivation, observe simpler structures**.

Zooming out: the key general question

To start with, focus on general non-stationary process.

Questions:

- Find Gaussian $Z_1, \ldots, Z_n$ such that $\max_{1 \le i \le n} |\sum_{j=1}^{i} (X_i - Z_i)| \approx$ small with optimal rate.
- Understand the exact covariance structure of $Z_1, \ldots, Z_n$.

Non-stationary structure$\approx$ arbitrary dependence! **For motivation, observe simpler structures**.

Zooming out: the key general question

To start with, focus on general non-stationary process.

Questions:

- Find Gaussian $Z_1, \ldots, Z_n$ such that $\max_{1 \le i \le n} |\sum_{j=1}^{i} (X_i - Z_i)| \approx$ small with optimal rate.
- Understand the exact covariance structure of $Z_1, \ldots, Z_n$.

Non-stationary structure$\approx$ arbitrary dependence! **For motivation, observe simpler structures**.

Zooming out: the key general question

To start with, focus on general non-stationary process.

Questions:

- Find Gaussian $Z_1, \ldots, Z_n$ such that $\max_{1 \le i \le n} |\sum_{j=1}^{i} (X_i - Z_i)| \approx$ small with optimal rate.
- Understand the exact covariance structure of $Z_1, \ldots, Z_n$.

Non-stationary structure$\approx$ arbitrary dependence! **For motivation, observe simpler structures**.

Partial Answer for i.i.d.

Z. Wahrscheinlichkeitstheorie verw. Gebiete 32, 111–131 (1975)
© by Springer-Verlag 1975

An Approximation of Partial Sums of Independent RV'-s, and the Sample DF. I

J. Komlós, P. Major and G. Tusnády

Summary. Let $S_n = X_1 + X_2 + \cdots + X_n$ be the sum of i.i.d.r.v.-s, $EX_1 = 0$, $EX_1^2 = 1$, and let $T_n = Y_1 + Y_2 + \cdots + Y_n$ be the sum of independent standard normal variables. Strassen proved in [14] that if X_1 has a finite fourth moment, then there are appropriate versions of S_n and T_n (which, of course, are far from being independent) such that $|S_n - T_n| = O(n^{\frac{1}{4}}(\log n)^{\frac{1}{2}}(\log\log n)^{\frac{1}{4}})$ with probability one. A theorem of Bártfai [1] indicates that even if X_1 has a finite moment generating function, the best possible bound for any version of S_n, T_n is $O(\log n)$. In this paper we introduce a new construction for the pair S_n, T_n, and prove that if X_1 has a finite moment generating function, and satisfies condition i) or ii) of Theorem 1, then $|S_n - T_n| = O(\log n)$ with probability one for the constructed S_n, T_n. Our method will be applicable for the approximation of sample DF., too.

Z. Wahrscheinlichkeitstheorie verw. Gebiete
34, 33–58 (1976)

Zeitschrift für
Wahrscheinlichkeitstheorie
und verwandte Gebiete
© by Springer-Verlag 1976

An Approximation of Partial Sums of Independent RV's, and the Sample DF. II

J. Komlós, P. Major and G. Tusnády
Mathematical Institute of the Hungarian Academy of Sciences
1053 Budapest, Reáltonada u. 13–15, Hungary

Given a d.f. $F(x)$ with the property

$$\int xF(dx) = 0, \quad \int x^2F(dx) = 1, \quad \int e^{tx}F(dx) < \infty$$

for $|t| < t_0$, we construct a sequence $X_1, X_2, \ldots$ of i.i.d.r.v.'s with d.f. $F(x)$ and $Y_1, Y_2, \ldots$ with standard normal distribution in such a way that the sequences $S_n = X_1 + \cdots + X_n$, $T_n = Y_1 + \cdots + Y_n$, $n = 1, 2, \ldots$ satisfy the relation $|S_n - T_n| = O(\log n)$ a.s. Under some mild conditions the best possible normal approximation of the process S_n will also be given in the case when F has not a moment generating function.

- For i.i.d.: KMT (Komlós Major Tusnády) construction.
- Widely popular in *time-series*. Often such strong approximation results are called KMT theory.

Strong approximation for non-stationary process

- $S_i = \sum_{j=1}^{i} X_j$.
- $\mathbb{E}[X_i] = 0$.
- $\sup_i \mathbb{E}[|X_i|^p] < \infty$, $p > 2$.

- If i.i.d./stationary:

$$\max_{1 \le i \le n} |S_i - \sigma \mathbb{B}(i)| = o_{\mathbb{P}}(n^{1/p}), \ \mathbb{B} : \text{Brownian motion.}$$

  Here $\sigma^2 = \sum_{k \in \mathbb{Z}} \mathbb{E}[X_0 X_k]$ $(= \mathbb{E}[X_0^2]$ for i.i.d.)

- Bottleneck: for non-stationarity, no notion of uniform scale capturing dependence.
- Note: $\mathbb{E}[S_n^2/n] \to \sigma^2$.
- Potentially replace $\sigma \mathbb{B}(i)$ by Gaussian G_i, where $\mathrm{Cov}(G_i) = \mathrm{Cov}(S_i)$?

Strong approximation for non-stationary process

- $S_i = \sum_{j=1}^{i} X_j$.
- $\mathbb{E}[X_i] = 0$.
- $\sup_i \mathbb{E}[|X_i|^p] < \infty$, $p > 2$.

- If i.i.d./stationary:

$$\max_{1 \le i \le n} |S_i - \sigma \mathbb{B}(i)| = o_{\mathbb{P}}(n^{1/p}), \ \mathbb{B} : \text{Brownian motion.}$$

  Here $\sigma^2 = \sum_{k \in \mathbb{Z}} \mathbb{E}[X_0 X_k]$ $(= \mathbb{E}[X_0^2]$ for i.i.d.)

- Bottleneck: for non-stationarity, no notion of uniform scale capturing dependence.
- Note: $\mathbb{E}[S_n^2/n] \to \sigma^2$.
- Potentially replace $\sigma \mathbb{B}(i)$ by Gaussian G_i, where $\mathrm{Cov}(G_i) = \mathrm{Cov}(S_i)$?

Strong approximation for non-stationary process

- $S_i = \sum_{j=1}^{i} X_j$.
- $\mathbb{E}[X_i] = 0$.
- $\sup_i \mathbb{E}[|X_i|^p] < \infty$, $p > 2$.

- If i.i.d./stationary:

$$\max_{1 \le i \le n} |S_i - \sigma \mathbb{B}(i)| = o_{\mathbb{P}}(n^{1/p}), \ \mathbb{B} : \text{Brownian motion.}$$

  Here $\sigma^2 = \sum_{k \in \mathbb{Z}} \mathbb{E}[X_0 X_k]$ $(= \mathbb{E}[X_0^2]$ for i.i.d.$)$

- Bottleneck: for non-stationarity, no notion of uniform scale capturing dependence.
- Note: $\mathbb{E}[S_n^2/n] \to \sigma^2$.
- Potentially replace $\sigma \mathbb{B}(i)$ by Gaussian G_i, where $\mathrm{Cov}(G_i) = \mathrm{Cov}(S_i)$?

Strong approximation for non-stationary process

- $S_i = \sum_{j=1}^{i} X_j$.
- $\mathbb{E}[X_i] = 0$.
- $\sup_i \mathbb{E}[|X_i|^p] < \infty$, $p > 2$.

- If i.i.d./stationary:

$$\max_{1 \le i \le n} |S_i - \sigma \mathbb{B}(i)| = o_{\mathbb{P}}(n^{1/p}), \ \mathbb{B} : \text{Brownian motion.}$$

  Here $\sigma^2 = \sum_{k \in \mathbb{Z}} \mathbb{E}[X_0 X_k]$ $(= \mathbb{E}[X_0^2]$ for i.i.d.)

- Bottleneck: for non-stationarity, no notion of uniform scale capturing dependence.
- Note: $\mathbb{E}[S_n^2/n] \to \sigma^2$.
- Potentially replace $\sigma \mathbb{B}(i)$ by Gaussian G_i, where $\mathrm{Cov}(G_i) = \mathrm{Cov}(S_i)$?

A Potential Result

How do we get hold of Gaussian G_i's such that $\mathrm{Cov}(G_i) = \mathrm{Cov}(S_i)$?

Let $Z_1, \ldots, Z_n$ be jointly Gaussian with $\mathrm{Cov}(Z_i, Z_j) = \mathrm{Cov}(X_i, X_j)$, then define

$$G_i = \sum_{j=1}^{i} Z_j \implies \mathrm{Cov}(G_i) = \mathrm{Cov}(S_i), \ i \in [n].$$

Key Question

Can we find $Z_1, \ldots, Z_n$ be jointly Gaussian with $\mathrm{Cov}(Z_i, Z_j) = \mathrm{Cov}(X_i, X_j)$, such that

$$\max_{1 \le i \le n} |\sum_{j=1}^{i} (X_j - Z_j)| = o_{\mathbb{P}}(n^{1/p})?$$

To formally prove this, we need to mathematically track the non-stationarity.

A Potential Result

How do we get hold of Gaussian G_i's such that $\mathrm{Cov}(G_i) = \mathrm{Cov}(S_i)$?

Let $Z_1, \ldots, Z_n$ be jointly Gaussian with $\mathrm{Cov}(Z_i, Z_j) = \mathrm{Cov}(X_i, X_j)$, then define

$$G_i = \sum_{j=1}^{i} Z_j \implies \mathrm{Cov}(G_i) = \mathrm{Cov}(S_i), \ i \in [n].$$

Key Question

Can we find $Z_1, \ldots, Z_n$ be jointly Gaussian with $\mathrm{Cov}(Z_i, Z_j) = \mathrm{Cov}(X_i, X_j)$, such that

$$\max_{1 \le i \le n} |\sum_{j=1}^{i} (X_j - Z_j)| = o_{\mathbb{P}}(n^{1/p})?$$

To formally prove this, we need to mathematically track the non-stationarity.

A Potential Result

How do we get hold of Gaussian G_i's such that $\mathrm{Cov}(G_i) = \mathrm{Cov}(S_i)$?

Let $Z_1, \ldots, Z_n$ be jointly Gaussian with $\mathrm{Cov}(Z_i, Z_j) = \mathrm{Cov}(X_i, X_j)$, then define

$$G_i = \sum_{j=1}^{i} Z_j \implies \mathrm{Cov}(G_i) = \mathrm{Cov}(S_i), \ i \in [n].$$

Key Question

Can we find $Z_1, \ldots, Z_n$ be jointly Gaussian with $\mathrm{Cov}(Z_i, Z_j) = \mathrm{Cov}(X_i, X_j)$, such that

$$\max_{1 \le i \le n} |\sum_{j=1}^{i} (X_j - Z_j)| = o_{\mathbb{P}}(n^{1/p})?$$

To formally prove this, we need to mathematically track the non-stationarity.

A Potential Result

How do we get hold of Gaussian G_i's such that $\mathrm{Cov}(G_i) = \mathrm{Cov}(S_i)$?

Let $Z_1, \ldots, Z_n$ be jointly Gaussian with $\mathrm{Cov}(Z_i, Z_j) = \mathrm{Cov}(X_i, X_j)$, then define

$$G_i = \sum_{j=1}^{i} Z_j \implies \mathrm{Cov}(G_i) = \mathrm{Cov}(S_i), \ i \in [n].$$

Key Question

Can we find $Z_1, \ldots, Z_n$ be jointly Gaussian with $\mathrm{Cov}(Z_i, Z_j) = \mathrm{Cov}(X_i, X_j)$, such that

$$\max_{1 \le i \le n} |\sum_{j=1}^{i} (X_j - Z_j)| = o_{\mathbb{P}}(n^{1/p})?$$

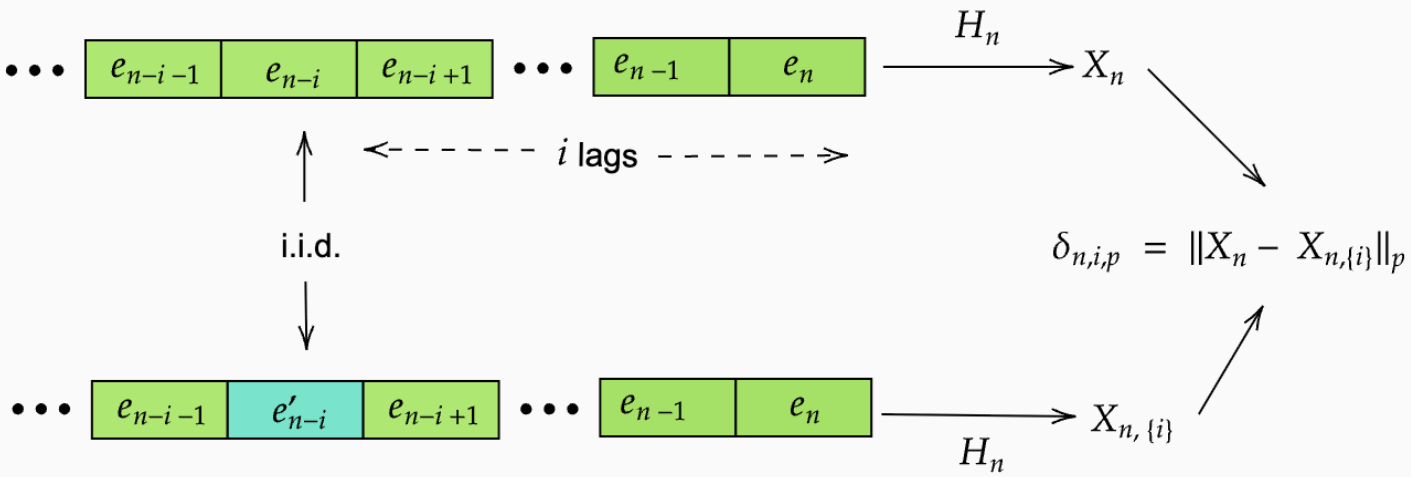
To formally prove this, we need to mathematically track the non-stationarity.

How to deal with non-stationarity?

$X_i = H_i(e_i, e_{i-1}, \ldots)$, $\{e_i\}_{i\in\mathbb{Z}}$, i.i.d "innovations" or "generators".

Notations and Assumptions

$\|X\|_p := (\mathbb{E}[|X|^p])^{1/p}$, $\sup_i \mathbb{E}[|X_i|^p] < \infty$, $p > 2$.



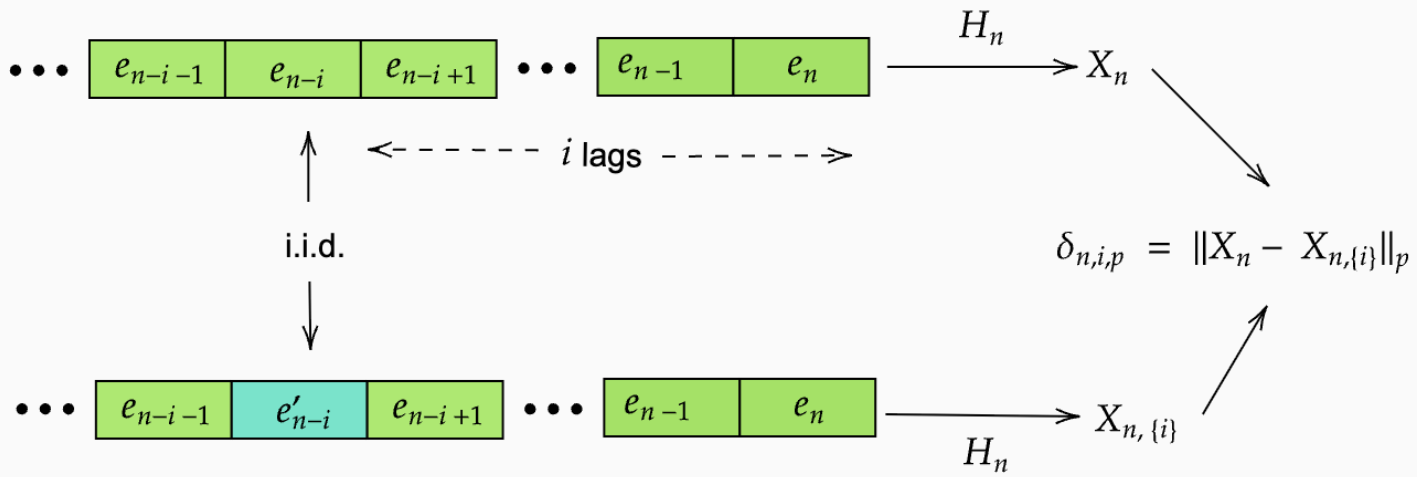
- *Functional dependence measure*(Wu (2005)) : $\delta_{i,p} = \sup_n \delta_{n,i,p}$.
- For linear process $X_i = \sum_{j=0} a_j e_{i-j}$, $\delta_{i,p} \propto |a_i|$, $p > 2$.
- Impose conditions on how $\delta_{j,p}$ decays.

How to deal with non-stationarity?

$X_i = H_i(e_i, e_{i-1}, \ldots)$, $\{e_i\}_{i\in\mathbb{Z}}$, i.i.d "innovations" or "generators".

Notations and Assumptions

$\|X\|_p := (\mathbb{E}[|X|^p])^{1/p}$, $\sup_i \mathbb{E}[|X_i|^p] < \infty$, $p > 2$.



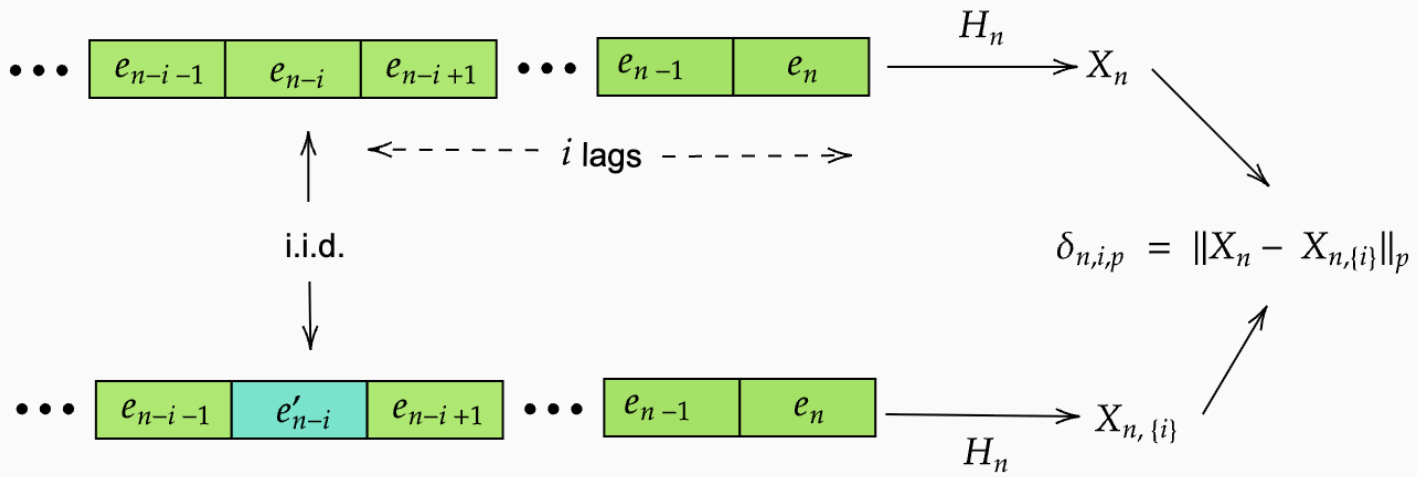
- *Functional dependence measure*(Wu (2005)) : $\delta_{i,p} = \sup_n \delta_{n,i,p}$.
- For linear process $X_i = \sum_{j=0} a_j e_{i-j}$, $\delta_{i,p} \propto |a_i|$, $p > 2$.
- Impose conditions on how $\delta_{j,p}$ decays.

How to deal with non-stationarity?

$X_i = H_i(e_i, e_{i-1}, \ldots)$, $\{e_i\}_{i\in\mathbb{Z}}$, i.i.d "innovations" or "generators".

Notations and Assumptions

$\|X\|_p := (\mathbb{E}[|X|^p])^{1/p}$, $\sup_i \mathbb{E}[|X_i|^p] < \infty$, $p > 2$.



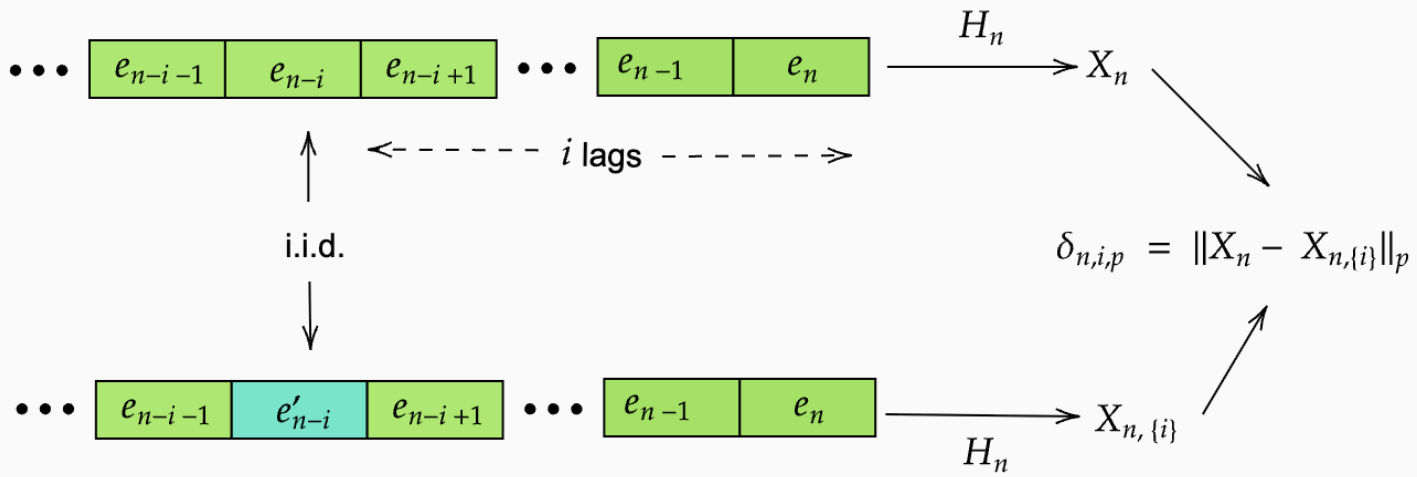
- *Functional dependence measure*(Wu (2005)) : $\delta_{i,p} = \sup_n \delta_{n,i,p}$.
- For linear process $X_i = \sum_{j=0} a_j e_{i-j}$, $\delta_{i,p} \propto |a_i|$, $p > 2$.
- Impose conditions on how $\delta_{j,p}$ decays.

How to deal with non-stationarity?

$X_i = H_i(e_i, e_{i-1}, \ldots)$, $\{e_i\}_{i\in\mathbb{Z}}$, i.i.d "innovations" or "generators".

Notations and Assumptions

$\|X\|_p := (\mathbb{E}[|X|^p])^{1/p}$, $\sup_i \mathbb{E}[|X_i|^p] < \infty$, $p > 2$.



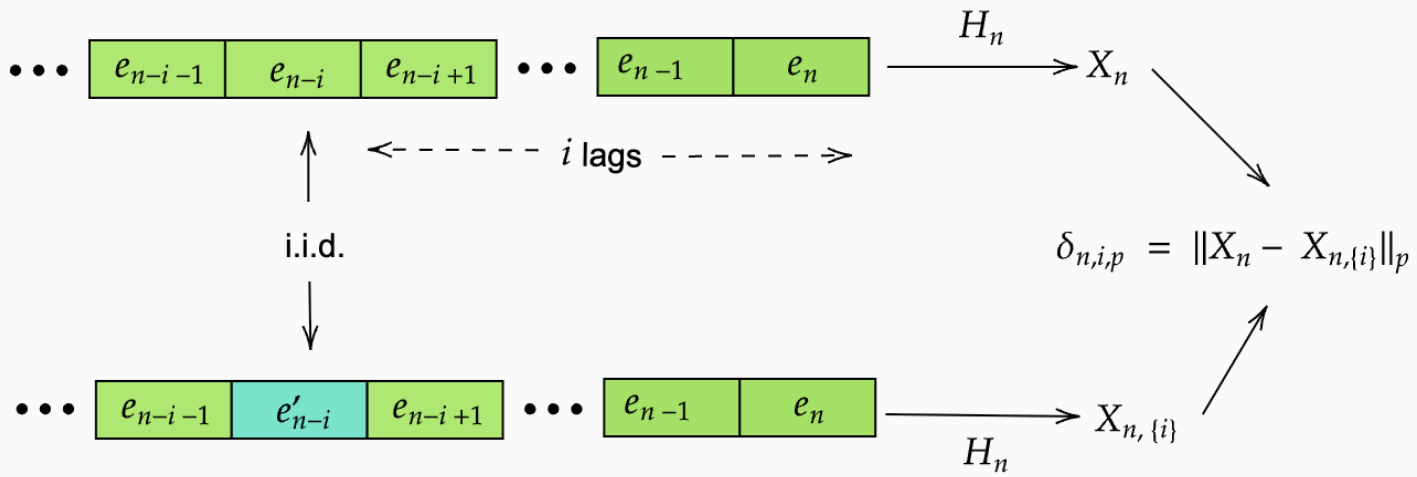
- *Functional dependence measure*(Wu (2005)) : $\delta_{i,p} = \sup_n \delta_{n,i,p}$.
- For linear process $X_i = \sum_{j=0} a_j e_{i-j}$, $\delta_{i,p} \propto |a_i|$, $p > 2$.
- Impose conditions on how $\delta_{j,p}$ decays.

How to deal with non-stationarity?

$X_i = H_i(e_i, e_{i-1}, \ldots)$, $\{e_i\}_{i\in\mathbb{Z}}$, i.i.d "innovations" or "generators".

Notations and Assumptions

$\|X\|_p := (\mathbb{E}[|X|^p])^{1/p}$, $\sup_i \mathbb{E}[|X_i|^p] < \infty$, $p > 2$.



- *Functional dependence measure*(Wu (2005)) : $\delta_{i,p} = \sup_n \delta_{n,i,p}$.
- For linear process $X_i = \sum_{j=0} a_j e_{i-j}$, $\delta_{i,p} \propto |a_i|$, $p > 2$.
- Impose conditions on how $\delta_{j,p}$ decays.

Remedy: Main Theorem

Decay conditions:

- Assume $\Delta_{m,p} := \sum_{j \geq m} \delta_{j,p} = O(m^{-A})$ for some $A > A_0(p) \approx (p/4 \vee 1)$.

Covariance matching approximations

Theorem (Informal, Thm 2.2 in B. et. al., AoS, 2024)
There exists Gaussian $Z_1, \ldots, Z_n$, $\mathrm{Cov}(Z_i, Z_j) = \mathrm{Cov}(X_i, X_j)$ such that

$$\max_{1 \leq j \leq n} \left| \sum_{i=1}^{j} (X_i - Z_i) \right| = o_{\mathbb{P}}(n^{1/p}).$$

- Useful for locally stationary/ cyclostationary processes.
- In particular, for stationary process, we can estimate the covariances.

Remedy: Main Theorem

Decay conditions:

- Assume $\Delta_{m,p} := \sum_{j \geq m} \delta_{j,p} = O(m^{-A})$ for some $A > A_0(p) \approx (p/4 \vee 1)$.

Covariance matching approximations

Theorem (Informal, Thm 2.2 in B. et. al., AoS, 2024)
There exists Gaussian $Z_1, \ldots, Z_n$, $\mathrm{Cov}(Z_i, Z_j) = \mathrm{Cov}(X_i, X_j)$ *such that*

$$\max_{1 \leq j \leq n} \left| \sum_{i=1}^{j} (X_i - Z_i) \right| = o_{\mathbb{P}}(n^{1/p}).$$

- Useful for locally stationary/ cyclostationary processes.
- In particular, for stationary process, we can estimate the covariances.

Remedy: Main Theorem

Decay conditions:

- Assume $\Delta_{m,p} := \sum_{j \geq m} \delta_{j,p} = O(m^{-A})$ for some $A > A_0(p) \approx (p/4 \vee 1)$.

Covariance matching approximations

Theorem (Informal, Thm 2.2 in B. et. al., AoS, 2024)
There exists Gaussian $Z_1, \ldots, Z_n$, $\mathrm{Cov}(Z_i, Z_j) = \mathrm{Cov}(X_i, X_j)$ *such that*

$$\max_{1 \leq j \leq n} \left| \sum_{i=1}^{j} (X_i - Z_i) \right| = o_{\mathbb{P}}(n^{1/p}).$$

- Useful for locally stationary/ cyclostationary processes.
- In particular, for stationary process, we can estimate the covariances.

Remedy: Main Theorem

Decay conditions:

- Assume $\Delta_{m,p} := \sum_{j \geq m} \delta_{j,p} = O(m^{-A})$ for some $A > A_0(p) \approx (p/4 \vee 1)$.

Covariance matching approximations

Theorem (Informal, Thm 2.2 in B. et. al., AoS, 2024)
There exists Gaussian $Z_1, \ldots, Z_n$, $\mathrm{Cov}(Z_i, Z_j) = \mathrm{Cov}(X_i, X_j)$ *such that*

$$\max_{1 \leq j \leq n} \left| \sum_{i=1}^{j} (X_i - Z_i) \right| = o_{\mathbb{P}}(n^{1/p}).$$

- Useful for locally stationary/ cyclostationary processes.
- In particular, for stationary process, we can estimate the covariances.

What's the benefit of covariance matching approximations?

- For non-stationary process, no notion of functional CLT due to no LRV.
- **For stationary process, we have other approximations! Why should we do this one?**
- Competitive Gaussian approximation for stationary process: $\max_{1\leq j\leq n}\left|\sum_{i=1}^{j} X_i - \sigma\mathbb{B}(j)\right| = o_{\mathbb{P}}(n^{1/p})$.
- **Turns out, covariance matching approximations are much more accurate in finite samples.**

What's the benefit of covariance matching approximations?

- For non-stationary process, no notion of functional CLT due to no LRV.
- **For stationary process, we have other approximations! Why should we do this one?**
- Competitive Gaussian approximation for stationary process: $\max_{1\leq j\leq n}\left|\sum_{i=1}^{j} X_i - \sigma\mathbb{B}(j)\right| = o_{\mathbb{P}}(n^{1/p})$.
- **Turns out, covariance matching approximations are much more accurate in finite samples.**

What's the benefit of covariance matching approximations?

- For non-stationary process, no notion of functional CLT due to no LRV.
- **For stationary process, we have other approximations! Why should we do this one?**
- Competitive Gaussian approximation for stationary process: $\max_{1\leq j\leq n}\left|\sum_{i=1}^{j} X_i - \sigma\mathbb{B}(j)\right| = o_{\mathbb{P}}(n^{1/p})$.
- **Turns out, covariance matching approximations are much more accurate in finite samples.**

Finite sample accuracy

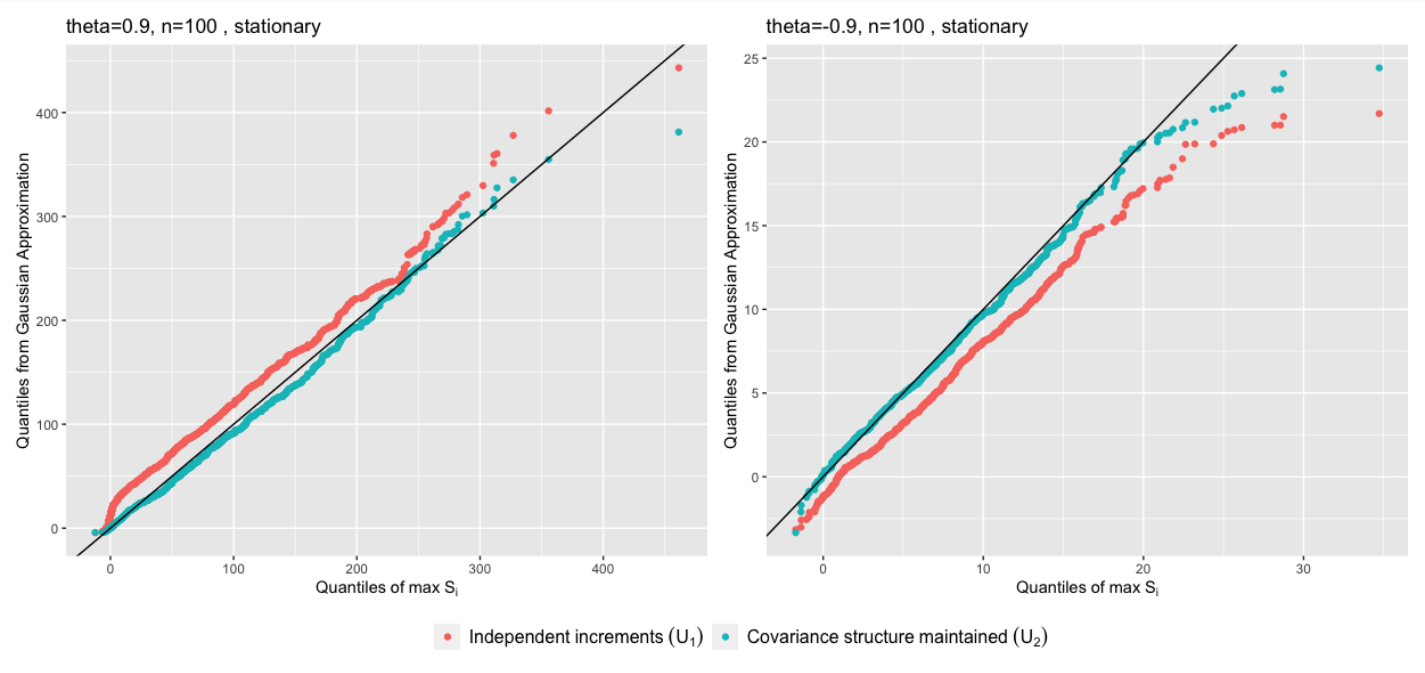


Figure 2: AR(1) model with autocorrelation $\theta = 0.9, -0.9.$, errors $\overset{\text{i.i.d.}}{\sim} t_4/\sqrt{2}$.

Good news

Turns out, these are also the most general way to establish uniform control for the particular cases of SGD and local SGD!

Finite sample accuracy

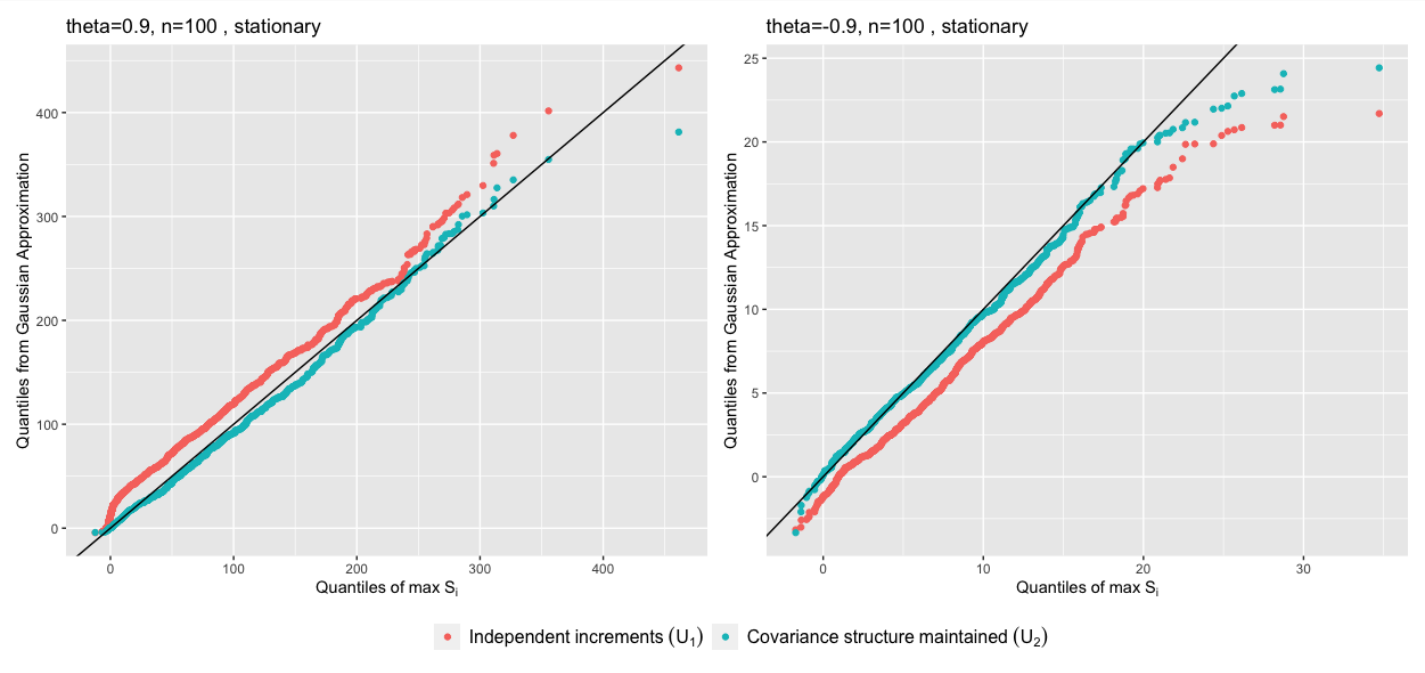


Figure 2: AR(1) model with autocorrelation $\theta = 0.9, -0.9.$, errors $\overset{\text{i.i.d.}}{\sim} t_4/\sqrt{2}$.

Good news

Turns out, these are also the most general way to establish uniform control for the particular cases of SGD and local SGD!

Recap and Way Forward

- What is Uniform control for local SGD? $\rightarrow$ What is uniform control for SGD? (Easier).
- SGD is like a non-stationary time-series.
- Established uniform control via *covariance-matching approximations* for non-stationary time-series.
- **Can we apply them directly to SGD?** NO.

Recap and Way Forward

- What is Uniform control for local SGD? $\rightarrow$ What is uniform control for SGD? (Easier).
- SGD is like a non-stationary time-series.
- Established uniform control via *covariance-matching approximations* for non-stationary time-series.
- **Can we apply them directly to SGD?** NO.

Recap and Way Forward

- What is Uniform control for local SGD? $\rightarrow$ What is uniform control for SGD? (Easier).
- SGD is like a non-stationary time-series.
- Established uniform control via *covariance-matching approximations* for non-stationary time-series.
- **Can we apply them directly to SGD?** NO.

Recap and Way Forward

- What is Uniform control for local SGD? $\rightarrow$ What is uniform control for SGD? (Easier).
- SGD is like a non-stationary time-series.
- Established uniform control via *covariance-matching approximations* for non-stationary time-series.
- **Can we apply them directly to SGD?** NO.

Recap and Way Forward

- What is Uniform control for local SGD? $\rightarrow$ What is uniform control for SGD? (Easier).
- SGD is like a non-stationary time-series.
- Established uniform control via *covariance-matching approximations* for non-stationary time-series.
- **Can we apply them directly to SGD?** NO.

Zooming in: the particular case of SGD

SGD iterate: $\theta_t = \theta_{t-1} - \eta_t \nabla_\theta f(\theta_{t-1}, \xi_t)$.

- SGD iterates doesn't generally satisfy the fast-decay conditions.
  Condition: $\sum_{j \geq m} \delta_{j,p} = O(m^{-A}) \implies \delta_{i,p} \to 0$ as $i \to \infty$.
  However, for SGD in many problems, $\delta_{i,p} = \sup_n \delta_{n,i,p} \asymp C$.
- **δ's are analogue of linear coefficient for linear process.**
  For linear process $X_n = \sum_{i=0} a_i e_{n-i}$, $\delta_{n,p} \propto |a_n|$, $p > 2$, $p > 2$.
- A different route: **Can we linearize SGD and work with it directly?**

Zooming in: the particular case of SGD

SGD iterate: $\theta_t = \theta_{t-1} - \eta_t \nabla_\theta f(\theta_{t-1}, \xi_t)$.

- SGD iterates doesn't generally satisfy the fast-decay conditions.
  Condition: $\sum_{j \geq m} \delta_{j,p} = O(m^{-A}) \implies \delta_{i,p} \to 0$ as $i \to \infty$.
  However, for SGD in many problems, $\delta_{i,p} = \sup_n \delta_{n,i,p} \asymp C$.
- **δ's are analogue of linear coefficient for linear process.**
  For linear process $X_n = \sum_{i=0} a_i e_{n-i}$, $\delta_{n,p} \propto |a_n|$, $p > 2$, $p > 2$.
- A different route: **Can we linearize SGD and work with it directly?**

Zooming in: the particular case of SGD

SGD iterate: $\theta_t = \theta_{t-1} - \eta_t \nabla_\theta f(\theta_{t-1}, \xi_t)$.

- SGD iterates doesn't generally satisfy the fast-decay conditions.
  Condition: $\sum_{j \geq m} \delta_{j,p} = O(m^{-A}) \implies \delta_{i,p} \to 0$ as $i \to \infty$.
  However, for SGD in many problems, $\delta_{i,p} = \sup_n \delta_{n,i,p} \asymp C$.
- **δ's are analogue of linear coefficient for linear process.**
  For linear process $X_n = \sum_{i=0} a_i e_{n-i}$, $\delta_{n,p} \propto |a_n|$, $p > 2$, $p > 2$.
- A different route: **Can we linearize SGD and work with it directly?**

Zooming in: the particular case of SGD

SGD iterate: $\theta_t = \theta_{t-1} - \eta_t \nabla_\theta f(\theta_{t-1}, \xi_t)$.

- SGD iterates doesn't generally satisfy the fast-decay conditions.
  Condition: $\sum_{j \geq m} \delta_{j,p} = O(m^{-A}) \implies \delta_{i,p} \to 0$ as $i \to \infty$.
  However, for SGD in many problems, $\delta_{i,p} = \sup_n \delta_{n,i,p} \asymp C$.
- **δ's are analogue of linear coefficient for linear process.**
  For linear process $X_n = \sum_{i=0} a_i e_{n-i}$, $\delta_{n,p} \propto |a_n|$, $p > 2$, $p > 2$.
- A different route: **Can we linearize SGD and work with it directly?**

Zooming in: the particular case of SGD

SGD iterate: $\theta_t = \theta_{t-1} - \eta_t \nabla_\theta f(\theta_{t-1}, \xi_t)$.

- SGD iterates doesn't generally satisfy the fast-decay conditions.
  Condition: $\sum_{j \geq m} \delta_{j,p} = O(m^{-A}) \implies \delta_{i,p} \to 0$ as $i \to \infty$.
  However, for SGD in many problems, $\delta_{i,p} = \sup_n \delta_{n,i,p} \asymp C$.
- δ's are analogue of linear coefficient for linear process.
  For linear process $X_n = \sum_{i=0} a_i e_{n-i}$, $\delta_{n,p} \propto |a_n|$, $p > 2$, $p > 2$.
- A different route: **Can we linearize SGD and work with it directly?**

Zooming in: the particular case of SGD

SGD iterate: $\theta_t = \theta_{t-1} - \eta_t \nabla_\theta f(\theta_{t-1}, \xi_t)$.

- SGD iterates doesn't generally satisfy the fast-decay conditions.
  Condition: $\sum_{j \geq m} \delta_{j,p} = O(m^{-A}) \implies \delta_{i,p} \to 0$ as $i \to \infty$.
  However, for SGD in many problems, $\delta_{i,p} = \sup_n \delta_{n,i,p} \asymp C$.
- **δ's are analogue of linear coefficient for linear process.**
  For linear process $X_n = \sum_{i=0} a_i e_{n-i}$, $\delta_{n,p} \propto |a_n|$, $p > 2$, $p > 2$.
- A different route: **Can we linearize SGD and work with it directly?**

Zooming in: the particular case of SGD

SGD iterate: $\theta_t = \theta_{t-1} - \eta_t \nabla_\theta f(\theta_{t-1}, \xi_t)$.

- SGD iterates doesn't generally satisfy the fast-decay conditions.
  Condition: $\sum_{j \geq m} \delta_{j,p} = O(m^{-A}) \implies \delta_{i,p} \to 0$ as $i \to \infty$.
  However, for SGD in many problems, $\delta_{i,p} = \sup_n \delta_{n,i,p} \asymp C$.
- **δ's are analogue of linear coefficient for linear process.**
  For linear process $X_n = \sum_{i=0} a_i e_{n-i}$, $\delta_{n,p} \propto |a_n|$, $p > 2$, $p > 2$.
- A different route: **Can we linearize SGD and work with it directly?**

Invariance principles for SGD

Goal of SGD: $\hat{\theta} = \arg\min_\theta \sum_{i=1}^n f(\theta, \xi_i)$.

Notations

- $\theta^\star = \arg\min_\theta F(\theta)$, where $F(\theta) = \mathbb{E}[f(\theta, \xi)]$. Essentially $\hat{\theta} \xrightarrow{\mathbb{P}} \theta^\star$.
- $A := \nabla_2 F(\theta^\star) \gtrsim I$ (strong convexity).
- Let gradient noise: $g(\theta, \xi) = \nabla F(\theta) - \nabla f(\theta, \xi)$.

- **SGD to linearized SGD**:

$$
\begin{aligned}
\theta_t &= \theta_{t-1} - \eta_t \nabla f(\theta_{t-1}, \xi_t) \\
&= \theta_{t-1} - \eta_t \nabla F(\theta_{t-1}) + \eta_t g(\theta_{t-1}, \xi_t) && \text{(plug-in gradient noise)} \\
&\approx \theta_{t-1} - \eta_t A(\theta_{t-1} - \theta^\star) + \eta_t g(\theta_{t-1}, \xi_t) && \text{(Taylor Series)} \\
\implies \theta_t - \theta^\star &\approx (I - \eta_t A)(\theta_{t-1} - \theta^\star) + \eta_t g(\theta_{t-1}, \xi_t). && \textbf{(Linearized SGD)}
\end{aligned}
$$

Invariance principles for SGD

Goal of SGD: $\hat{\theta} = \arg\min_\theta \sum_{i=1}^n f(\theta, \xi_i)$.

Notations

- $\theta^\star = \arg\min_\theta F(\theta)$, where $F(\theta) = \mathbb{E}[f(\theta, \xi)]$. Essentially $\hat{\theta} \xrightarrow{\mathbb{P}} \theta^\star$.
- $A := \nabla_2 F(\theta^\star) \gtrsim I$ (strong convexity).
- Let gradient noise: $g(\theta, \xi) = \nabla F(\theta) - \nabla f(\theta, \xi)$.

- **SGD to linearized SGD**:

$$
\begin{aligned}
\theta_t &= \theta_{t-1} - \eta_t \nabla f(\theta_{t-1}, \xi_t) \\
&= \theta_{t-1} - \eta_t \nabla F(\theta_{t-1}) + \eta_t g(\theta_{t-1}, \xi_t) && \text{(plug-in gradient noise)} \\
&\approx \theta_{t-1} - \eta_t A(\theta_{t-1} - \theta^\star) + \eta_t g(\theta_{t-1}, \xi_t) && \text{(Taylor Series)} \\
\implies \theta_t - \theta^\star &\approx (I - \eta_t A)(\theta_{t-1} - \theta^\star) + \eta_t g(\theta_{t-1}, \xi_t). && \textbf{(Linearized SGD)}
\end{aligned}
$$

Invariance principles for SGD

Goal of SGD: $\hat{\theta} = \arg\min_\theta \sum_{i=1}^n f(\theta, \xi_i)$.

Notations

- $\theta^\star = \arg\min_\theta F(\theta)$, where $F(\theta) = \mathbb{E}[f(\theta, \xi)]$. Essentially $\hat{\theta} \xrightarrow{\mathbb{P}} \theta^\star$.
- $A := \nabla_2 F(\theta^\star) \gtrsim I$ (strong convexity).
- Let gradient noise: $g(\theta, \xi) = \nabla F(\theta) - \nabla f(\theta, \xi)$.

- **SGD to linearized SGD**:

$$
\begin{aligned}
\theta_t &= \theta_{t-1} - \eta_t \nabla f(\theta_{t-1}, \xi_t) \\
&= \theta_{t-1} - \eta_t \nabla F(\theta_{t-1}) + \eta_t g(\theta_{t-1}, \xi_t) && \text{(plug-in gradient noise)} \\
&\approx \theta_{t-1} - \eta_t A(\theta_{t-1} - \theta^\star) + \eta_t g(\theta_{t-1}, \xi_t) && \text{(Taylor Series)} \\
\implies \theta_t - \theta^\star &\approx (I - \eta_t A)(\theta_{t-1} - \theta^\star) + \eta_t g(\theta_{t-1}, \xi_t). && \textbf{(Linearized SGD)}
\end{aligned}
$$

Invariance principles for SGD

Goal of SGD: $\hat{\theta} = \arg\min_\theta \sum_{i=1}^n f(\theta, \xi_i)$.

Notations

- $\theta^\star = \arg\min_\theta F(\theta)$, where $F(\theta) = \mathbb{E}[f(\theta, \xi)]$. Essentially $\hat{\theta} \xrightarrow{\mathbb{P}} \theta^\star$.
- $A := \nabla_2 F(\theta^\star) \gtrsim I$ (strong convexity).
- Let gradient noise: $g(\theta, \xi) = \nabla F(\theta) - \nabla f(\theta, \xi)$.

- **SGD to linearized SGD**:

$$
\begin{aligned}
\theta_t &= \theta_{t-1} - \eta_t \nabla f(\theta_{t-1}, \xi_t) \\
&= \theta_{t-1} - \eta_t \nabla F(\theta_{t-1}) + \eta_t g(\theta_{t-1}, \xi_t) && \text{(plug-in gradient noise)} \\
&\approx \theta_{t-1} - \eta_t A(\theta_{t-1} - \theta^\star) + \eta_t g(\theta_{t-1}, \xi_t) && \text{(Taylor Series)} \\
\implies \theta_t - \theta^\star &\approx (I - \eta_t A)(\theta_{t-1} - \theta^\star) + \eta_t g(\theta_{t-1}, \xi_t). && \textbf{(Linearized SGD)}
\end{aligned}
$$

Invariance principles for SGD

Goal of SGD: $\hat{\theta} = \arg\min_\theta \sum_{i=1}^n f(\theta, \xi_i)$.

Notations

- $\theta^\star = \arg\min_\theta F(\theta)$, where $F(\theta) = \mathbb{E}[f(\theta, \xi)]$. Essentially $\hat{\theta} \xrightarrow{\mathbb{P}} \theta^\star$.
- $A := \nabla_2 F(\theta^\star) \gtrsim I$ (strong convexity).
- Let gradient noise: $g(\theta, \xi) = \nabla F(\theta) - \nabla f(\theta, \xi)$.

- **SGD to linearized SGD**:

$$
\begin{aligned}
\theta_t &= \theta_{t-1} - \eta_t \nabla f(\theta_{t-1}, \xi_t) \\
&= \theta_{t-1} - \eta_t \nabla F(\theta_{t-1}) + \eta_t g(\theta_{t-1}, \xi_t) && \text{(plug-in gradient noise)} \\
&\approx \theta_{t-1} - \eta_t A(\theta_{t-1} - \theta^\star) + \eta_t g(\theta_{t-1}, \xi_t) && \text{(Taylor Series)} \\
\implies \theta_t - \theta^\star &\approx (I - \eta_t A)(\theta_{t-1} - \theta^\star) + \eta_t g(\theta_{t-1}, \xi_t). && \textbf{(Linearized SGD)}
\end{aligned}
$$

Invariance principles for SGD

Goal of SGD: $\hat{\theta} = \arg\min_\theta \sum_{i=1}^n f(\theta, \xi_i)$.

Notations

- $\theta^\star = \arg\min_\theta F(\theta)$, where $F(\theta) = \mathbb{E}[f(\theta, \xi)]$. Essentially $\hat{\theta} \xrightarrow{\mathbb{P}} \theta^\star$.
- $A := \nabla_2 F(\theta^\star) \gtrsim I$ (strong convexity).
- Let gradient noise: $g(\theta, \xi) = \nabla F(\theta) - \nabla f(\theta, \xi)$.

- **SGD to linearized SGD**:

$$\begin{aligned}
\theta_t &= \theta_{t-1} - \eta_t \nabla f(\theta_{t-1}, \xi_t) \\
&= \theta_{t-1} - \eta_t \nabla F(\theta_{t-1}) + \eta_t g(\theta_{t-1}, \xi_t) && \text{(plug-in gradient noise)} \\
&\approx \theta_{t-1} - \eta_t A(\theta_{t-1} - \theta^\star) + \eta_t g(\theta_{t-1}, \xi_t) && \text{(Taylor Series)} \\
\implies \theta_t - \theta^\star &\approx (I - \eta_t A)(\theta_{t-1} - \theta^\star) + \eta_t g(\theta_{t-1}, \xi_t). && \textbf{(Linearized SGD)}
\end{aligned}$$

Invariance principles for SGD

Goal of SGD: $\hat{\theta} = \arg\min_\theta \sum_{i=1}^n f(\theta, \xi_i)$.

Notations

- $\theta^\star = \arg\min_\theta F(\theta)$, where $F(\theta) = \mathbb{E}[f(\theta, \xi)]$. Essentially $\hat{\theta} \xrightarrow{\mathbb{P}} \theta^\star$.
- $A := \nabla_2 F(\theta^\star) \gtrsim I$ (strong convexity).
- Let gradient noise: $g(\theta, \xi) = \nabla F(\theta) - \nabla f(\theta, \xi)$.

- **SGD to linearized SGD**:

$$
\begin{aligned}
\theta_t &= \theta_{t-1} - \eta_t \nabla f(\theta_{t-1}, \xi_t) \\
&= \theta_{t-1} - \eta_t \nabla F(\theta_{t-1}) + \eta_t g(\theta_{t-1}, \xi_t) && \text{(plug-in gradient noise)} \\
&\approx \theta_{t-1} - \eta_t A(\theta_{t-1} - \theta^\star) + \eta_t g(\theta_{t-1}, \xi_t) && \text{(Taylor Series)} \\
\implies \theta_t - \theta^\star &\approx (I - \eta_t A)(\theta_{t-1} - \theta^\star) + \eta_t g(\theta_{t-1}, \xi_t). && \textbf{(Linearized SGD)}
\end{aligned}
$$

Invariance principles for SGD

Goal of SGD: $\hat{\theta} = \arg\min_\theta \sum_{i=1}^n f(\theta, \xi_i)$.

Notations

- $\theta^\star = \arg\min_\theta F(\theta)$, where $F(\theta) = \mathbb{E}[f(\theta, \xi)]$. Essentially $\hat{\theta} \xrightarrow{\mathbb{P}} \theta^\star$.
- $A := \nabla_2 F(\theta^\star) \gtrsim I$ (strong convexity).
- Let gradient noise: $g(\theta, \xi) = \nabla F(\theta) - \nabla f(\theta, \xi)$.

- **SGD to linearized SGD**:

$$\begin{aligned}
\theta_t &= \theta_{t-1} - \eta_t \nabla f(\theta_{t-1}, \xi_t) \\
&= \theta_{t-1} - \eta_t \nabla F(\theta_{t-1}) + \eta_t g(\theta_{t-1}, \xi_t) && \text{(plug-in gradient noise)} \\
&\approx \theta_{t-1} - \eta_t A(\theta_{t-1} - \theta^\star) + \eta_t g(\theta_{t-1}, \xi_t) && \text{(Taylor Series)}
\end{aligned}$$

$$\implies \theta_t - \theta^\star \approx (I - \eta_t A)(\theta_{t-1} - \theta^\star) + \eta_t g(\theta_{t-1}, \xi_t). \textbf{ (Linearized SGD)}$$

Invariance principles for SGD

Goal of SGD: $\hat{\theta} = \arg\min_\theta \sum_{i=1}^n f(\theta, \xi_i)$.

Notations

- $\theta^\star = \arg\min_\theta F(\theta)$, where $F(\theta) = \mathbb{E}[f(\theta, \xi)]$. Essentially $\hat{\theta} \xrightarrow{\mathbb{P}} \theta^\star$.
- $A := \nabla_2 F(\theta^\star) \gtrsim I$ (strong convexity).
- Let gradient noise: $g(\theta, \xi) = \nabla F(\theta) - \nabla f(\theta, \xi)$.

- **SGD to linearized SGD**:

$$\begin{aligned}
\theta_t &= \theta_{t-1} - \eta_t \nabla f(\theta_{t-1}, \xi_t) \\
&= \theta_{t-1} - \eta_t \nabla F(\theta_{t-1}) + \eta_t g(\theta_{t-1}, \xi_t) && \text{(plug-in gradient noise)} \\
&\approx \theta_{t-1} - \eta_t A(\theta_{t-1} - \theta^\star) + \eta_t g(\theta_{t-1}, \xi_t) && \text{(Taylor Series)} \\
\implies \theta_t - \theta^\star &\approx (I - \eta_t A)(\theta_{t-1} - \theta^\star) + \eta_t g(\theta_{t-1}, \xi_t). && \textbf{(Linearized SGD)}
\end{aligned}$$

Invariance principles for SGD

Linearized SGD

$$\theta_t - \theta^\star = (I - \eta_t A)(\theta_{t-1} - \theta^\star) + \eta_t g(\theta_{t-1}, \xi_t).$$

Assume $\mathbb{E}[|g(\theta, \xi)|^p] < \infty$. Denote $S := \mathrm{Cov}(g(\theta^\star, \xi))$.

- **Covariance-matching approximation** (abridged from **B.** et. al., NeurIPS 2025):

Theorem

Assume smoothness conditions+strong convexity. There exists $\{Y_t\}$ given by $Y_t = (I - \eta_t A) Y_{t-1} + \eta_t Z_t$, $Z_t \sim N(0, S)$ i.i.d., such that with $\eta_t \asymp t^{-\beta}$, it holds

$$\max_{1 \le i \le n} |\sum_{j=1}^{i} (\theta_j - \theta^\star - Y_j)| = o_{\mathbb{P}}(n^{1-\beta}) + o_{\mathbb{P}}(n^{1/p}), \ \beta \in (1/2, 1).$$

Invariance principles for SGD

Linearized SGD

$$\theta_t - \theta^\star = (I - \eta_t A)(\theta_{t-1} - \theta^\star) + \eta_t g(\theta_{t-1}, \xi_t).$$

Assume $\mathbb{E}[|g(\theta, \xi)|^p] < \infty$. Denote $S := \mathrm{Cov}(g(\theta^\star, \xi))$.

- **Covariance-matching approximation** (abridged from **B**. et. al., NeurIPS 2025):

Theorem

Assume smoothness conditions+strong convexity. There exists $\{Y_t\}$ given by $Y_t = (I - \eta_t A) Y_{t-1} + \eta_t Z_t$, $Z_t \sim N(0, S)$ i.i.d., such that with $\eta_t \asymp t^{-\beta}$, it holds

$$\max_{1 \le i \le n} |\sum_{j=1}^{i} (\theta_j - \theta^\star - Y_j)| = o_{\mathbb{P}}(n^{1-\beta}) + o_{\mathbb{P}}(n^{1/p}), \ \beta \in (1/2, 1).$$

Invariance principles for SGD

Linearized SGD

$$\theta_t - \theta^\star = (I - \eta_t A)(\theta_{t-1} - \theta^\star) + \eta_t g(\theta_{t-1}, \xi_t).$$

Assume $\mathbb{E}[|g(\theta, \xi)|^p] < \infty$. Denote $S := \mathrm{Cov}(g(\theta^\star, \xi))$.

- **Covariance-matching approximation** (abridged from **B**. et. al., NeurIPS 2025):

Theorem

Assume smoothness conditions+strong convexity. There exists $\{Y_t\}$ given by $Y_t = (I - \eta_t A) Y_{t-1} + \eta_t Z_t$, $Z_t \sim N(0, S)$ i.i.d., such that with $\eta_t \asymp t^{-\beta}$, it holds

$$\max_{1 \le i \le n} |\sum_{j=1}^{i} (\theta_j - \theta^\star - Y_j)| = o_{\mathbb{P}}(n^{1-\beta}) + o_{\mathbb{P}}(n^{1/p}), \ \beta \in (1/2, 1).$$

Invariance principles for local SGD

Can we do something similar in Local SGD?

Yes! if we can rewrite local SGD as a modified SGD!

Invariance principles for local SGD

Can we do something similar in Local SGD?

Yes! if we can rewrite local SGD as a modified SGD!

Rewriting Local SGD

Total K many clients.

- Each client k can run their own *parallel*, SGD:

$$\theta_t^k = \theta_{t-1}^k - \eta_t K w_k \nabla f_k(\theta_{t-1}^k, \xi_t^k), \text{ no sharing happens at iteration } t.$$

- They also must collaborate $\Rightarrow$ share gradient with *neighbors* (through some underlying graph), *periodically* (say at time points $\tau, 2\tau, \ldots$).
- For example, if $N(l)$: neighbors of client/server l, then for $t \in \tau, 2\tau, \ldots,$

$$\theta_t^k = \sum_{l \in N(k)} \frac{1}{|N(l)|} (\theta_{t-1}^l - \eta_t K w_l \nabla f_l(\theta_{t-1}^l, \xi_t^l)).$$

- Let $Y_t = \frac{1}{K} \sum_{k=1}^K \theta_t^k$.
- Key observation:

$$Y_t = Y_{t-1} - \eta_t \sum_{k=1}^K w_k \nabla f_k(\theta_{t-1}^k, \xi_t^k).$$

Rewriting Local SGD

Total K many clients.

- Each client k can run their own *parallel*, SGD:

$$\theta_t^k = \theta_{t-1}^k - \eta_t K w_k \nabla f_k(\theta_{t-1}^k, \xi_t^k), \text{ no sharing happens at iteration } t.$$

- They also must collaborate $\Rightarrow$ share gradient with *neighbors* (through some underlying graph), *periodically* (say at time points $\tau, 2\tau, \ldots$).
- For example, if $N(l)$: neighbors of client/server l, then for $t \in \tau, 2\tau, \ldots,$

$$\theta_t^k = \sum_{l \in N(k)} \frac{1}{|N(l)|} (\theta_{t-1}^l - \eta_t K w_l \nabla f_l(\theta_{t-1}^l, \xi_t^l)).$$

- Let $Y_t = \frac{1}{K} \sum_{k=1}^K \theta_t^k$.
- Key observation:

$$Y_t = Y_{t-1} - \eta_t \sum_{k=1}^K w_k \nabla f_k(\theta_{t-1}^k, \xi_t^k).$$

Rewriting Local SGD

Total K many clients.

- Each client k can run their own *parallel*, SGD:

$$\theta_t^k = \theta_{t-1}^k - \eta_t K w_k \nabla f_k(\theta_{t-1}^k, \xi_t^k), \text{ no sharing happens at iteration } t.$$

- They also must collaborate $\Rightarrow$ share gradient with *neighbors* (through some underlying graph), *periodically* (say at time points $\tau, 2\tau, \ldots$).
- For example, if $N(l)$: neighbors of client/server l, then for $t \in \tau, 2\tau, \ldots,$

$$\theta_t^k = \sum_{l \in N(k)} \frac{1}{|N(l)|} (\theta_{t-1}^l - \eta_t K w_l \nabla f_l(\theta_{t-1}^l, \xi_t^l)).$$

- Let $Y_t = \frac{1}{K} \sum_{k=1}^K \theta_t^k$.
- Key observation:

$$Y_t = Y_{t-1} - \eta_t \sum_{k=1}^K w_k \nabla f_k(\theta_{t-1}^k, \xi_t^k).$$

Rewriting Local SGD

Total K many clients.

- Each client k can run their own *parallel*, SGD:

$$\theta_t^k = \theta_{t-1}^k - \eta_t K w_k \nabla f_k(\theta_{t-1}^k, \xi_t^k), \text{ no sharing happens at iteration } t.$$

- They also must collaborate $\Rightarrow$ share gradient with *neighbors* (through some underlying graph), *periodically* (say at time points $\tau, 2\tau, \ldots$).
- For example, if $N(l)$: neighbors of client/server l, then for $t \in \tau, 2\tau, \ldots,$

$$\theta_t^k = \sum_{l \in N(k)} \frac{1}{|N(l)|} (\theta_{t-1}^l - \eta_t K w_l \nabla f_l(\theta_{t-1}^l, \xi_t^l)).$$

- Let $Y_t = \frac{1}{K} \sum_{k=1}^K \theta_t^k$.
- Key observation:

$$Y_t = Y_{t-1} - \eta_t \sum_{k=1}^K w_k \nabla f_k(\theta_{t-1}^k, \xi_t^k).$$

Rewriting Local SGD

Total K many clients.

- Each client k can run their own *parallel*, SGD:

$$\theta_t^k = \theta_{t-1}^k - \eta_t K w_k \nabla f_k(\theta_{t-1}^k, \xi_t^k), \text{ no sharing happens at iteration } t.$$

- They also must collaborate $\Rightarrow$ share gradient with *neighbors* (through some underlying graph), *periodically* (say at time points $\tau, 2\tau, \ldots$).
- For example, if $N(l)$: neighbors of client/server l, then for $t \in \tau, 2\tau, \ldots,$

$$\theta_t^k = \sum_{l \in N(k)} \frac{1}{|N(l)|} (\theta_{t-1}^l - \eta_t K w_l \nabla f_l(\theta_{t-1}^l, \xi_t^l)).$$

- Let $Y_t = \frac{1}{K} \sum_{k=1}^K \theta_t^k$.
- Key observation:

$$Y_t = Y_{t-1} - \eta_t \sum_{k=1}^K w_k \nabla f_k(\theta_{t-1}^k, \xi_t^k).$$

Invariance principles for local SGD

$Y_s = \frac{1}{K}\sum_{k=1}^{K} \theta_s^k$. Define Y_t^G using linearization.

Thm 3.1 in B. et. al., *NeurIPS* 2025

With step-sizes $\eta_t \asymp t^{-\beta}$, $\beta \in (1/2, 1)$,

$$\max_{1 \le t \le n} \left| \sum_{s=1}^{t} (Y_s - \theta_K^\star - Y_s^G) \right| = O_{\mathbb{P}}(n^{1-\beta}) + o_{\mathbb{P}}(n^{1/p} K^{-1/2} \log n).$$

- For SGD, we got $O_{\mathbb{P}}(n^{1/p} + n^{1-\beta})$.
- $K^{-1/2}$ comes from averaging over the K clients/servers.

Invariance principles for local SGD

$Y_s = \frac{1}{K}\sum_{k=1}^{K} \theta_s^k$. Define Y_t^G using linearization.

Thm 3.1 in B. et. al., *NeurIPS* 2025

With step-sizes $\eta_t \asymp t^{-\beta}$, $\beta \in (1/2, 1)$,

$$\max_{1 \le t \le n} \left| \sum_{s=1}^{t} (Y_s - \theta_K^\star - Y_s^G) \right| = O_{\mathbb{P}}(n^{1-\beta}) + o_{\mathbb{P}}(n^{1/p} K^{-1/2} \log n).$$

- For SGD, we got $O_{\mathbb{P}}(n^{1/p} + n^{1-\beta})$.
- $K^{-1/2}$ comes from averaging over the K clients/servers.

Invariance principles for local SGD

$Y_s = \frac{1}{K}\sum_{k=1}^{K} \theta_s^k$. Define Y_t^G using linearization.

Thm 3.1 in B. et. al., *NeurIPS* 2025

With step-sizes $\eta_t \asymp t^{-\beta}$, $\beta \in (1/2, 1)$,

$$\max_{1 \le t \le n} \left| \sum_{s=1}^{t} (Y_s - \theta_K^\star - Y_s^G) \right| = O_{\mathbb{P}}(n^{1-\beta}) + o_{\mathbb{P}}(n^{1/p} K^{-1/2} \log n).$$

- For SGD, we got $O_{\mathbb{P}}(n^{1/p} + n^{1-\beta})$.
- $K^{-1/2}$ comes from averaging over the K clients/servers.

Recap and Way Forward

- Take the covariance matching idea for general non-stationary time-series.
- Prove similar approximation for SGD based on linearized SGD.
- Prove for local SGD based on corresponding linearization.

How to use this for the Byzantine Attack problem?

Recap and Way Forward

- Take the covariance matching idea for general non-stationary time-series.
- Prove similar approximation for SGD based on linearized SGD.
- Prove for local SGD based on corresponding linearization.

How to use this for the Byzantine Attack problem?

Recap and Way Forward

- Take the covariance matching idea for general non-stationary time-series.
- Prove similar approximation for SGD based on linearized SGD.
- Prove for local SGD based on corresponding linearization.

How to use this for the Byzantine Attack problem?

Recap and Way Forward

- Take the covariance matching idea for general non-stationary time-series.
- Prove similar approximation for SGD based on linearized SGD.
- Prove for local SGD based on corresponding linearization.

How to use this for the Byzantine Attack problem?

Application: Byzantine Attack detection

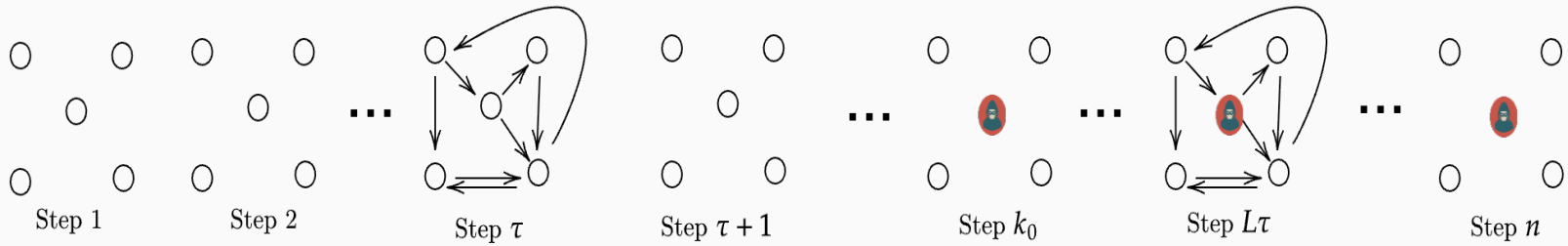


Figure 3: Byzantine Attack

- **Goal 1**: *Early stopping* $\rightarrow$ Sequential Testing.
- **Goal 2**: *Attack instance detection* $\rightarrow$ Estimate after sequential process stops.
- **Observation**: The company/central moderator can collect $Y_t = \frac{1}{K}\sum_{k=1}^{K} \theta_t^k$'s only during sharing/synchronization steps $t = \tau, 2\tau, \ldots$.

Work with $\{Y_s : s = \tau, 2\tau, \ldots\}$.

Application: Byzantine Attack detection

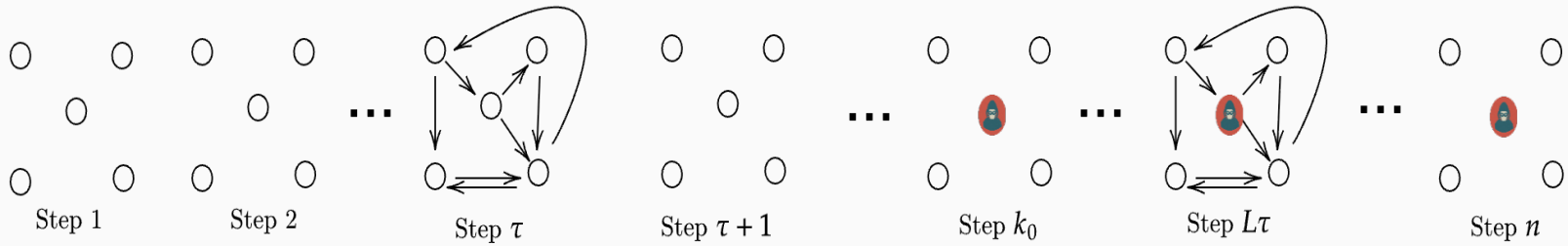


Figure 3: Byzantine Attack

- **Goal 1**: *Early stopping* $\rightarrow$ Sequential Testing.
- **Goal 2**: *Attack instance detection* $\rightarrow$ Estimate after sequential process stops.
- **Observation**: The company/central moderator can collect $Y_t = \frac{1}{K}\sum_{k=1}^{K} \theta_t^k$'s only during sharing/synchronization steps $t = \tau, 2\tau, \ldots$.

Work with $\{Y_s : s = \tau, 2\tau, \ldots\}$.

Application: Byzantine Attack detection

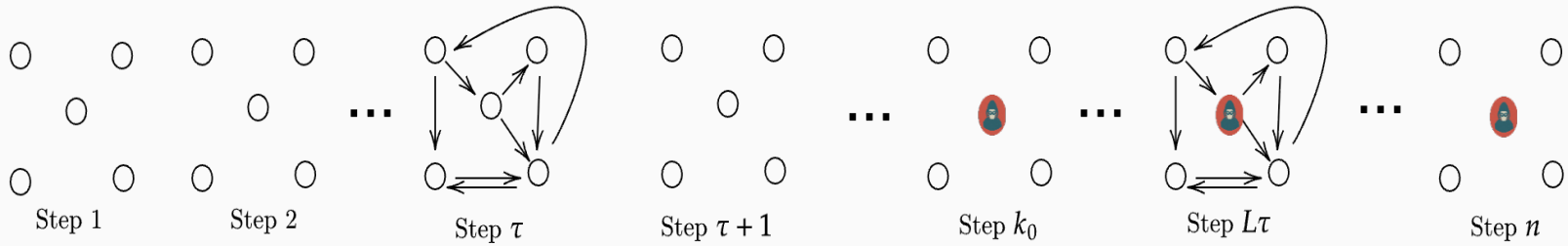


Figure 3: Byzantine Attack

- **Goal 1**: *Early stopping* $\rightarrow$ Sequential Testing.
- **Goal 2**: *Attack instance detection* $\rightarrow$ Estimate after sequential process stops.
- **Observation**: The company/central moderator can collect $Y_t = \frac{1}{K}\sum_{k=1}^{K} \theta_t^k$'s only during sharing/synchronization steps $t = \tau, 2\tau, \ldots$.

Work with $\{Y_s : s = \tau, 2\tau, \ldots\}$.

Attack detection

A wrapper algorithm:

For $L = 1, 2, \ldots,$

- At $L\tau$-th step: compute test-stat for detection: $f(Y_s : s \in \{\tau, 2\tau, \ldots, L\tau\})$.
- Compute corresponding Gaussian analogue: $f(Y_s^G : s \in \{\tau, 2\tau, \ldots, L\tau\})$.
- Stop at L if $f(Y_s : s \in \{\tau, 2\tau, \ldots, L\tau\}) > Q_{0.95}(f(Y_s^G : s \in \{\tau, 2\tau, \ldots, L\tau\}))$.
- Employ your favorite detection mechanism (CUSUM/MOSUM/WBS etc.) on $Y_s : s \in \{\tau, 2\tau, \ldots, L\tau\}$.
- Theoretical validity as long as f is Lipschitz.

Attack detection

A wrapper algorithm:

For $L = 1, 2, \ldots,$

- At $L\tau$-th step: compute test-stat for detection: $f(Y_s : s \in \{\tau, 2\tau, \ldots, L\tau\})$.
- Compute corresponding Gaussian analogue: $f(Y_s^G : s \in \{\tau, 2\tau, \ldots, L\tau\})$.
- Stop at L if $f(Y_s : s \in \{\tau, 2\tau, \ldots, L\tau\}) > Q_{0.95}(f(Y_s^G : s \in \{\tau, 2\tau, \ldots, L\tau\}))$.
- Employ your favorite detection mechanism (CUSUM/MOSUM/WBS etc.) on $Y_s : s \in \{\tau, 2\tau, \ldots, L\tau\}$.
- Theoretical validity as long as f is Lipschitz.

Attack detection

A wrapper algorithm:

For $L = 1, 2, \ldots,$

- At $L\tau$-th step: compute test-stat for detection: $f(Y_s : s \in \{\tau, 2\tau, \ldots, L\tau\})$.
- Compute corresponding Gaussian analogue: $f(Y_s^G : s \in \{\tau, 2\tau, \ldots, L\tau\})$.
- Stop at L if $f(Y_s : s \in \{\tau, 2\tau, \ldots, L\tau\}) > Q_{0.95}(f(Y_s^G : s \in \{\tau, 2\tau, \ldots, L\tau\}))$.
- Employ your favorite detection mechanism (CUSUM/MOSUM/WBS etc.) on $Y_s : s \in \{\tau, 2\tau, \ldots, L\tau\}$.
- Theoretical validity as long as f is Lipschitz.

Attack detection

A wrapper algorithm:

For $L = 1, 2, \ldots,$

- At $L\tau$-th step: compute test-stat for detection: $f(Y_s : s \in \{\tau, 2\tau, \ldots, L\tau\})$.
- Compute corresponding Gaussian analogue: $f(Y_s^G : s \in \{\tau, 2\tau, \ldots, L\tau\})$.
- Stop at L if $f(Y_s : s \in \{\tau, 2\tau, \ldots, L\tau\}) > Q_{0.95}(f(Y_s^G : s \in \{\tau, 2\tau, \ldots, L\tau\}))$.
- Employ your favorite detection mechanism (CUSUM/MOSUM/WBS etc.) on $Y_s : s \in \{\tau, 2\tau, \ldots, L\tau\}$.
- Theoretical validity as long as f is Lipschitz.

Attack detection

A wrapper algorithm:

For $L = 1, 2, \ldots,$

- At $L\tau$-th step: compute test-stat for detection: $f(Y_s : s \in \{\tau, 2\tau, \ldots, L\tau\})$.
- Compute corresponding Gaussian analogue: $f(Y_s^G : s \in \{\tau, 2\tau, \ldots, L\tau\})$.
- Stop at L if $f(Y_s : s \in \{\tau, 2\tau, \ldots, L\tau\}) > Q_{0.95}(f(Y_s^G : s \in \{\tau, 2\tau, \ldots, L\tau\}))$.
- Employ your favorite detection mechanism (CUSUM/MOSUM/WBS etc.) on $Y_s : s \in \{\tau, 2\tau, \ldots, L\tau\}$.
- Theoretical validity as long as f is Lipschitz.

Attack detection

A wrapper algorithm:

For $L = 1, 2, \ldots,$

- At $L\tau$-th step: compute test-stat for detection: $f(Y_s : s \in \{\tau, 2\tau, \ldots, L\tau\})$.
- Compute corresponding Gaussian analogue: $f(Y_s^G : s \in \{\tau, 2\tau, \ldots, L\tau\})$.
- Stop at L if $f(Y_s : s \in \{\tau, 2\tau, \ldots, L\tau\}) > Q_{0.95}(f(Y_s^G : s \in \{\tau, 2\tau, \ldots, L\tau\}))$.
- Employ your favorite detection mechanism (CUSUM/MOSUM/WBS etc.) on $Y_s : s \in \{\tau, 2\tau, \ldots, L\tau\}$.
- Theoretical validity as long as f is Lipschitz.

Attack detection

- Attack at iteration $t_0 = n/2$ with $K_0 = K/2$ many malevolent clients. Here, $n = 500$, $K = 10$, sync parameter $\tau = 20$.
- For early stopping, **CUSUM**:

$$\max_{1 \le s \le L} s|\bar{Y}_s - \bar{Y}_t|, \ \bar{Y}_s = \frac{1}{s}\sum_{t=1}^{s} Y_{s\tau}.$$

- Detection: $\arg\max_{1 \le s \le L} s|\bar{Y}_s - \bar{Y}_t|$.
- All the inference is based on $B = 500$ bootstrap samples.

Attack strength	Probability of detection	Stopping time $\hat{T}_0$ (mean, 95% CI)	Attack instance (mean, 95% CI)
0 (No attack)	0.046 (False positive)	–	–
0.5	0.172	400.67, (141.25, 496.875)	201.233, (58.75, 290.875)
1	0.966	412.49, (345.05, 482)	265.203, (144.05, 309)
1.5	1	389.61, (292.475, 470.525)	255.772, (155.9, 310.525)
2	1	356.32, (311.475, 397.525)	249.672, (118.85, 286.575)
2.5	1	343.39, (294.95, 379.525)	247.098, (113.8, 281)
3	1	334.57, (282.95, 367)	249.572, (94.275, 276)

Table 1: Simulation results of attack detection.

Attack detection

- Attack at iteration $t_0 = n/2$ with $K_0 = K/2$ many malevolent clients. Here, $n = 500$, $K = 10$, sync parameter $\tau = 20$.
- For early stopping, **CUSUM**:

$$\max_{1 \le s \le L} s|\bar{Y}_s - \bar{Y}_t|, \ \bar{Y}_s = \frac{1}{s}\sum_{t=1}^{s} Y_{s\tau}.$$

- Detection: $\arg\max_{1 \le s \le L} s|\bar{Y}_s - \bar{Y}_t|$.
- All the inference is based on $B = 500$ bootstrap samples.

Attack strength	Probability of detection	Stopping time $\hat{T}_0$ (mean, 95% CI)	Attack instance (mean, 95% CI)
0 (No attack)	0.046 (False positive)	–	–
0.5	0.172	400.67, (141.25, 496.875)	201.233, (58.75, 290.875)
1	0.966	412.49, (345.05, 482)	265.203, (144.05, 309)
1.5	1	389.61, (292.475, 470.525)	255.772, (155.9, 310.525)
2	1	356.32, (311.475, 397.525)	249.672, (118.85, 286.575)
2.5	1	343.39, (294.95, 379.525)	247.098, (113.8, 281)
3	1	334.57, (282.95, 367)	249.572, (94.275, 276)

Table 1: Simulation results of attack detection.

Attack detection

- Attack at iteration $t_0 = n/2$ with $K_0 = K/2$ many malevolent clients. Here, $n = 500$, $K = 10$, sync parameter $\tau = 20$.
- For early stopping, **CUSUM**:

$$\max_{1 \le s \le L} s|\bar{Y}_s - \bar{Y}_t|, \ \bar{Y}_s = \frac{1}{s}\sum_{t=1}^{s} Y_{s\tau}.$$

- Detection: $\arg\max_{1 \le s \le L} s|\bar{Y}_s - \bar{Y}_t|$.
- All the inference is based on $B = 500$ bootstrap samples.

Attack strength	Probability of detection	Stopping time $\hat{T}_0$ (mean, 95% CI)	Attack instance (mean, 95% CI)
0 (No attack)	0.046 (False positive)	–	–
0.5	0.172	400.67, (141.25, 496.875)	201.233, (58.75, 290.875)
1	0.966	412.49, (345.05, 482)	265.203, (144.05, 309)
1.5	1	389.61, (292.475, 470.525)	255.772, (155.9, 310.525)
2	1	356.32, (311.475, 397.525)	249.672, (118.85, 286.575)
2.5	1	343.39, (294.95, 379.525)	247.098, (113.8, 281)
3	1	334.57, (282.95, 367)	249.572, (94.275, 276)

Table 1: Simulation results of attack detection.

Attack detection

- Attack at iteration $t_0 = n/2$ with $K_0 = K/2$ many malevolent clients. Here, $n = 500$, $K = 10$, sync parameter $\tau = 20$.
- For early stopping, **CUSUM**:

$$\max_{1 \le s \le L} s|\bar{Y}_s - \bar{Y}_t|, \ \bar{Y}_s = \frac{1}{s}\sum_{t=1}^{s} Y_{s\tau}.$$

- Detection: $\arg\max_{1 \le s \le L} s|\bar{Y}_s - \bar{Y}_t|$.
- All the inference is based on $B = 500$ bootstrap samples.

Attack strength	Probability of detection	Stopping time $\hat{T}_0$ (mean, 95% CI)	Attack instance (mean, 95% CI)
0 (No attack)	0.046 (False positive)	–	–
0.5	0.172	400.67, (141.25, 496.875)	201.233, (58.75, 290.875)
1	0.966	412.49, (345.05, 482)	265.203, (144.05, 309)
1.5	1	389.61, (292.475, 470.525)	255.772, (155.9, 310.525)
2	1	356.32, (311.475, 397.525)	249.672, (118.85, 286.575)
2.5	1	343.39, (294.95, 379.525)	247.098, (113.8, 281)
3	1	334.57, (282.95, 367)	249.572, (94.275, 276)

Table 1: Simulation results of attack detection.

Final Recap

- Byzantine attack on federated learning system: a pertinent problem in ML.
- Local SGD or SGD are particular form of non-stationary time-series.
- Establish *covariance-matching* uniform approximations for a general class of non-stationary process satisfying some decay conditions.
- SGD/local SGD does not satisfy decay conditions.
  Establish covariance-matching uniform approximations based on *linearized* versions.
- Provide a bootstrap-based sequential testing and attack instance estimation algorithm for the Byzantine attack problem.
- *Tools from time-series/broader statistics are immensely useful in many problems in modern ML/AI.*

Final Recap

- Byzantine attack on federated learning system: a pertinent problem in ML.
- Local SGD or SGD are particular form of non-stationary time-series.
- Establish *covariance-matching* uniform approximations for a general class of non-stationary process satisfying some decay conditions.
- SGD/local SGD does not satisfy decay conditions.
  Establish covariance-matching uniform approximations based on *linearized* versions.
- Provide a bootstrap-based sequential testing and attack instance estimation algorithm for the Byzantine attack problem.
- *Tools from time-series/broader statistics are immensely useful in many problems in modern ML/AI.*

Final Recap

- Byzantine attack on federated learning system: a pertinent problem in ML.
- Local SGD or SGD are particular form of non-stationary time-series.
- Establish *covariance-matching* uniform approximations for a general class of non-stationary process satisfying some decay conditions.
- SGD/local SGD does not satisfy decay conditions.
  Establish covariance-matching uniform approximations based on *linearized* versions.
- Provide a bootstrap-based sequential testing and attack instance estimation algorithm for the Byzantine attack problem.
- *Tools from time-series/broader statistics are immensely useful in many problems in modern ML/AI.*

Final Recap

- Byzantine attack on federated learning system: a pertinent problem in ML.
- Local SGD or SGD are particular form of non-stationary time-series.
- Establish *covariance-matching* uniform approximations for a general class of non-stationary process satisfying some decay conditions.
- SGD/local SGD does not satisfy decay conditions.
  Establish covariance-matching uniform approximations based on *linearized* versions.
- Provide a bootstrap-based sequential testing and attack instance estimation algorithm for the Byzantine attack problem.
- *Tools from time-series/broader statistics are immensely useful in many problems in modern ML/AI.*

Final Recap

- Byzantine attack on federated learning system: a pertinent problem in ML.
- Local SGD or SGD are particular form of non-stationary time-series.
- Establish *covariance-matching* uniform approximations for a general class of non-stationary process satisfying some decay conditions.
- SGD/local SGD does not satisfy decay conditions.
  Establish covariance-matching uniform approximations based on *linearized* versions.
- Provide a bootstrap-based sequential testing and attack instance estimation algorithm for the Byzantine attack problem.
- *Tools from time-series/broader statistics are immensely useful in many problems in modern ML/AI.*

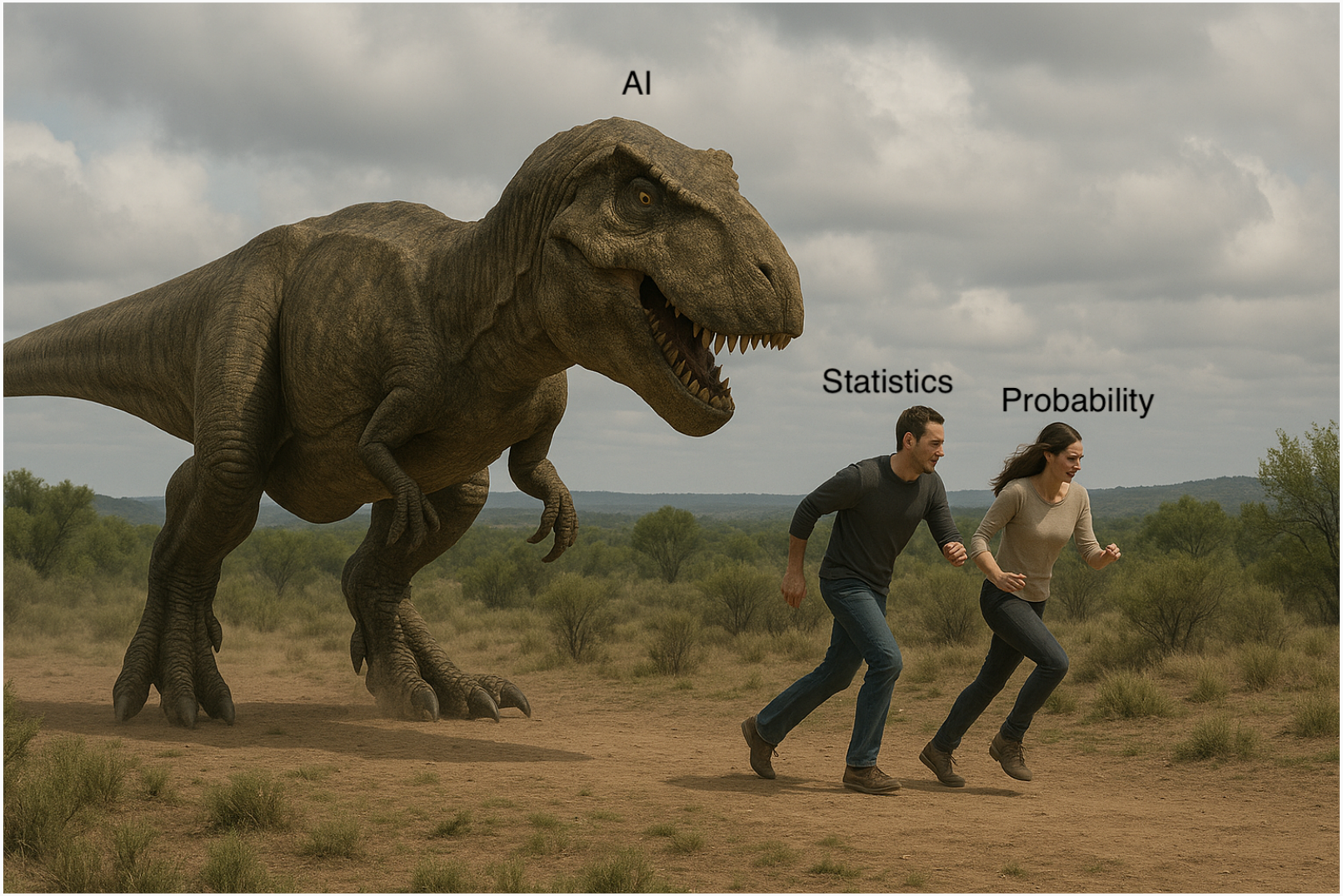
Final Recap

- Byzantine attack on federated learning system: a pertinent problem in ML.
- Local SGD or SGD are particular form of non-stationary time-series.
- Establish *covariance-matching* uniform approximations for a general class of non-stationary process satisfying some decay conditions.
- SGD/local SGD does not satisfy decay conditions.
  Establish covariance-matching uniform approximations based on *linearized* versions.
- Provide a bootstrap-based sequential testing and attack instance estimation algorithm for the Byzantine attack problem.
- *Tools from time-series/broader statistics are immensely useful in many problems in modern ML/AI.*

Final Recap

- Byzantine attack on federated learning system: a pertinent problem in ML.
- Local SGD or SGD are particular form of non-stationary time-series.
- Establish *covariance-matching* uniform approximations for a general class of non-stationary process satisfying some decay conditions.
- SGD/local SGD does not satisfy decay conditions.
  Establish covariance-matching uniform approximations based on *linearized* versions.
- Provide a bootstrap-based sequential testing and attack instance estimation algorithm for the Byzantine attack problem.
- *Tools from time-series/broader statistics are immensely useful in many problems in modern ML/AI.*

Perception



Possibility



More Results/Future research

- Federated learning. **Who are the malicious clients?**- already identified as a change-point problem in Qian et al., 2024.
- Uniform approximations in Q-learning and other reinforcement learning algorithms. **B.**, Lou, and Wu, 2025
- Testing if the change-points in each co-ordinate of a multivariate stationary time series, are synchronized. **B.** et al., 2025
- Conditional independence testing. Wieck-Sosa, Haddad, and Ramdas, 2025
- Watermark detection in LLM-generated texts. **B.**, Roy and Karmakar, 2025.

More Results/Future research

- Federated learning. **Who are the malicious clients?**- already identified as a change-point problem in Qian et al., 2024.
- Uniform approximations in Q-learning and other reinforcement learning algorithms. **B.**, Lou, and Wu, 2025
- Testing if the change-points in each co-ordinate of a multivariate stationary time series, are synchronized. **B.** et al., 2025
- Conditional independence testing. Wieck-Sosa, Haddad, and Ramdas, 2025
- Watermark detection in LLM-generated texts. **B.**, Roy and Karmakar, 2025.

More Results/Future research

- Federated learning. **Who are the malicious clients?**- already identified as a change-point problem in Qian et al., 2024.
- Uniform approximations in Q-learning and other reinforcement learning algorithms. **B.**, Lou, and Wu, 2025
- Testing if the change-points in each co-ordinate of a multivariate stationary time series, are synchronized. **B.** et al., 2025
- Conditional independence testing. Wieck-Sosa, Haddad, and Ramdas, 2025
- Watermark detection in LLM-generated texts. **B.**, Roy and Karmakar, 2025.

More Results/Future research

- Federated learning. **Who are the malicious clients?**- already identified as a change-point problem in Qian et al., 2024.
- Uniform approximations in Q-learning and other reinforcement learning algorithms. **B.**, Lou, and Wu, 2025
- Testing if the change-points in each co-ordinate of a multivariate stationary time series, are synchronized. **B.** et al., 2025
- Conditional independence testing. Wieck-Sosa, Haddad, and Ramdas, 2025
- Watermark detection in LLM-generated texts. **B.**, Roy and Karmakar, 2025.

More Results/Future research

- Federated learning. **Who are the malicious clients?**- already identified as a change-point problem in Qian et al., 2024.
- Uniform approximations in Q-learning and other reinforcement learning algorithms. **B.**, Lou, and Wu, 2025
- Testing if the change-points in each co-ordinate of a multivariate stationary time series, are synchronized. **B.** et al., 2025
- Conditional independence testing. Wieck-Sosa, Haddad, and Ramdas, 2025
- Watermark detection in LLM-generated texts. **B.**, Roy and Karmakar, 2025.

More Results/Future research

- Federated learning. **Who are the malicious clients?**- already identified as a change-point problem in Qian et al., 2024.
- Uniform approximations in Q-learning and other reinforcement learning algorithms. **B.**, Lou, and Wu, 2025
- Testing if the change-points in each co-ordinate of a multivariate stationary time series, are synchronized. **B.** et al., 2025
- Conditional independence testing. Wieck-Sosa, Haddad, and Ramdas, 2025
- Watermark detection in LLM-generated texts. **B.**, Roy and Karmakar, 2025.

References

 Blanchard, Peva et al. (2017). **"Machine learning with adversaries: byzantine tolerant gradient descent".** In: *Proceedings of the 31st International Conference on Neural Information Processing Systems*. NIPS'17. Long Beach, California, USA: Curran Associates Inc., pp. 118–128. ISBN: 9781510860964.

 Bottou, Léon (2010). **"Large-scale machine learning with stochastic gradient descent".** In: *Proceedings of COMPSTAT'2010: 19th International Conference on Computational StatisticsParis France, August 22-27, 2010 Keynote, Invited and Contributed Papers*. Springer, pp. 177–186.

 Chen, Mingzhe and Shuguang Cui (2024). **Communication Efficient Federated Learning for Wireless Networks.** Springer.

 Chen, Yiqiang et al. (2020). **"FedHealth: A Federated Transfer Learning Framework for Wearable Healthcare".** In: *IEEE Intelligent Systems* 35.4, pp. 83–93. DOI: 10.1109/MIS.2020.2988604.

 Data, Deepesh and Suhas Diggavi (2021). **"Byzantine-resilient high-dimensional SGD with local iterations on heterogeneous data".** In: *International Conference on Machine Learning*. PMLR, pp. 2478–2488.

 Du, Mengxuan et al. (2022). **"Decentralized federated learning with Markov chain based consensus for industrial IoT networks".** In: *IEEE Transactions on Industrial Informatics* 19.4, pp. 6006–6015.

 Duchi, John, Elad Hazan, and Yoram Singer (2011). **"Adaptive subgradient methods for online learning and stochastic optimization.".** In: *Journal of machine learning research* 12.7.

 Fan, Hongbin, Changbing Huang, and Yining Liu (2022). **"Federated learning-based privacy-preserving data aggregation scheme for IIoT".** In: *IEEE Access* 11, pp. 6700–6707.

 He, Chen, Yao Jin, and Jia Chen (2021). **"FedGen: Federated Text Generation with Attribute Constraints".** In: *Findings of the Association for Computational Linguistics: ACL-IJCNLP 2021*, pp. 1855–1861.

 Ji, An et al. (2025). **"Private Federated Learning In Real World Application–A Case Study".** In: *arXiv preprint arXiv:2502.04565*.

 Karimireddy, Sai Praneeth et al. (2020). **"SCAFFOLD: Stochastic controlled averaging for federated learning".** In: *International Conference on Machine Learning (ICML)*. PMLR, pp. 5132–5143.

 Kingma, Diederik P (2014). **"Adam: A method for stochastic optimization".** In: *arXiv preprint arXiv:1412.6980*.

 Koloskova, Anastasia et al. (2020). **"A Unified Theory of Decentralized SGD with Changing Topology and Local Updates".** In: *Proceedings of the 37th International Conference on Machine Learning, ICML 2020, 13-18 July 2020, Virtual Event*. Vol. 119. Proceedings of Machine Learning Research. PMLR, pp. 5381–5393. URL: http://proceedings.mlr.press/v119/koloskova20a.html.

- LeCun, Yann et al. (2002). **"Efficient backprop".** In: *Neural networks: Tricks of the trade*. Springer, pp. 9–50.
- Li, Tian et al. (2020). **"Federated optimization in heterogeneous networks".** In: *Proceedings of Machine Learning and Systems*. Vol. 2, pp. 429–450.
- McMahan, H Brendan et al. (2017). **"Communication-efficient learning of deep networks from decentralized data".** In: *Artificial Intelligence and Statistics*. PMLR, pp. 1273–1282.
- Nesterov, Yu. E. (1983). **"A method for solving the convex programming problem with convergence rate $O(1/k^2)$".** In: *Dokl. Akad. Nauk SSSR* 269.3, pp. 543–547. ISSN: 0002-3264.
- Poljak, B. T. (1964). **"Some methods of speeding up the convergence of iterative methods".** In: *Ž. Vyčisl. Mat i Mat. Fiz.* 4, pp. 791–803. ISSN: 0044-4669.


Qian, Chengde et al. (21–27 Jul 2024). **"ByMI: Byzantine Machine Identification with False Discovery Rate Control".** In: *Proceedings of the 41st International Conference on Machine Learning (ICML 2024)*. Ed. by Ruslan Salakhutdinov et al. Vol. 235. Proceedings of Machine Learning Research. PMLR, pp. 41357–41382. URL: https://proceedings.mlr.press/v235/qian24b.html.


Stich, Sebastian U. (2019). **"Local SGD Converges Fast and Communicates Little".** In: *7th International Conference on Learning Representations, ICLR 2019, New Orleans, LA, USA, May 6-9, 2019*. OpenReview.net. URL: https://openreview.net/forum?id=S1g2JnRcFX.


Tu, Jiyuan et al. (2021). **"Variance reduced median-of-means estimator for Byzantine-robust distributed inference".** In: *Journal of Machine Learning Research* 22.84, pp. 1–67.


Wieck-Sosa, Michael, Michel FC Haddad, and Aaditya Ramdas (2025). **"Conditional independence testing with a single realization of a multivariate nonstationary nonlinear time series".** In: *arXiv preprint arXiv:2504.21647*.

 Wu, Wei Biao (2005). **"Nonlinear system theory: another look at dependence".** In: *Proc. Natl. Acad. Sci. USA* 102.40, pp. 14150–14154. ISSN: 0027-8424,1091-6490. DOI: 10.1073/pnas.0506715102. URL: https://doi.org/10.1073/pnas.0506715102.

 Yang, Qiang et al. (2019). **"Federated machine learning: Concept and applications".** In: *ACM Transactions on Intelligent Systems and Technology (TIST)* 10.2, pp. 1–19.

 Yin, Dong et al. (2018). **"Byzantine-Robust Distributed Learning: Towards Optimal Statistical Rates".** In: *Proceedings of the 35th International Conference on Machine Learning, ICML 2018, Stockholmsmässan, Stockholm, Sweden, July 10-15, 2018*. Ed. by Jennifer G. Dy and Andreas Krause. Vol. 80. Proceedings of Machine Learning Research. PMLR, pp. 5636–5645. URL: http://proceedings.mlr.press/v80/yin18a.html.

Thank You!

Questions??

Appendix I: Generalizability of non-stationary representation

Example:

X_i is ARIMA$(1,1,0)$ if $X_i - X_{i-1} = Y_i = \sum_{k=0} \rho^k e_{i-k}$ can be re-written:

$$X_i = \sum_{j=-\infty}^{i} \sum_{k=0}^{\infty} \rho^k e_{i-k} = \sum_{j=-\infty}^{i} (\sum_{k=0}^{i-j} \rho^k) e_j = H_i(e_i, e_{i-1}, \ldots,).$$

Appendix I: Generalizability of non-stationary representation

Example:

X_i is ARIMA$(1,1,0)$ if $X_i - X_{i-1} = Y_i = \sum_{k=0} \rho^k e_{i-k}$ can be re-written:

$$X_i = \sum_{j=-\infty}^{i} \sum_{k=0}^{\infty} \rho^k e_{i-k} = \sum_{j=-\infty}^{i} (\sum_{k=0}^{i-j} \rho^k) e_j = H_i(e_i, e_{i-1}, \ldots,).$$

Appendix II: why is covariance-matching better?

No solid answer yet beyond mathematical details! But an educated guess!

- Brownian motion: independent increments, destroys covariance structure.
- Covariance matching: finite-sample exact covariances. So better results despite equal asymptotic rates.

Appendix II: why is covariance-matching better?

No solid answer yet beyond mathematical details! But an educated guess!

- Brownian motion: independent increments, destroys covariance structure.
- Covariance matching: finite-sample exact covariances. So better results despite equal asymptotic rates.

Appendix II: why is covariance-matching better?

No solid answer yet beyond mathematical details! But an educated guess!

- Brownian motion: independent increments, destroys covariance structure.
- Covariance matching: finite-sample exact covariances. So better results despite equal asymptotic rates.

Appendix III: Theoretical guarantees of Byzantine attack algorithm

- To detect the attack-instamce t_0, consider the CUSUM statistic $R_t := \max_{1\le s\le t}\, s\,|\bar{Y}_s - \bar{Y}_t|$.
- Gaussian process $\{G_t\}$ satisfies uniform approximation

$$\max_{1\le t\le n} |t\bar{Y}_t - G_t| = o_{\mathbb{P}}(n^{1/[]}).$$

- Let the Gaussian analogue be

$$R_t^G := \max_{1\le s\le t} |G_s - \tfrac{s}{t} G_t|.$$

- Then the approximation error satisfies

$$n^{-1/2} \max_{1\le t\le n} |R_t - R_t^G| = o_{\mathbb{P}}(1).$$

- For a suitable $a_n \downarrow 0$,

$$\mathbb{P}\big(R_t > Q_{1-\alpha}(R_t^G) + a_n \text{ for some } t\big) \;\le\; \alpha + o(1).$$

Appendix IV: The formal algorithm

The algorithm is as follows.

Algorithm 2 `Time-uniform Gaussian bootstrap`

Input: Initializations $\mathbf{\Theta}_0 = (\theta_0^1, \ldots, \theta_0^K) \in \mathbb{R}^{d\times K}$; Connection matrix $\mathbf{C}$; Synchronization parameter τ; Loss functions $f_k(\cdot, \xi^k), \xi^k \sim \mathcal{P}_k, k \in [K]$, weights $\{w_k\}_{k=1}^K$, number of iterations n, step-size schedules $\{\eta_t\}_{t=1}^n$, Hessian A; number of bootstrap samples B; covariance matrix $\mathcal{V}_K$.

- Let $E_\tau = \{\tau, 2\tau, \ldots, L\tau\}$, where $L = \lfloor \frac{n}{\tau} \rfloor$. Initialize $t = 1$. Stopping time $\hat{T}_0 = 1$, estimated attack instance $\hat{s}_0 = +\inf$.
- While $t \leq n$:
  1. Store the local SGD iterates Y_t, and calculate $R_t = \max_{1\leq s\leq t} s|\bar{Y}_s - \bar{Y}_t|$ and $s_t = \arg\max_{1\leq s\leq t} s|\bar{Y}_s - \bar{Y}_t|$.
  2. For $B = 1, \ldots, B$:
     Draw $Z_t^{(b)} \sim N(0, \mathcal{V}_K)$, and do $Y_{t,1}^{G,(b)} = (I - \eta_t A)Y_{t-1,1}^{G,(b)} + \eta_t Z_t^{(b)} K^{-1/2}, Y_{0,1}^{G,(b)} = \mathbf{0}$.
     Calculate $R_t^{G,(b)} = \max_{1\leq s\leq t} s|\bar{Y}_{s,1}^{G,(b)} - \bar{Y}_{t,1}^{G,(b)}|$
- $\hat{q}_{1-\alpha} \leftarrow$ sample quantile$(\{R_t^{G,(b)}\})$.
- **Thresholding**: If $R_t > \hat{q}_{1-\alpha} + c\sqrt{n}$:
  $\hat{T}_0 \leftarrow t$, $\hat{s}_0 \leftarrow s_{\hat{T}_0}$. Stop.
  Else $t+ = 1$.

Output: $\hat{T}_0 I\{\hat{T}_0 < n\}$, $s_{\hat{T}_0}$.